AUSTRALIAN NATIONAL UNIVERSITY

# COMP8620 Part B
# 完整深度课程讲义

从随机 MDP 到单机 Value Iteration 引导的联合 MCTS：实现、实验和理论边界

八讲完整体系·数学推导·代码逐段解析·实验统计·口试问题

默认：单机有限时域 Value Iteration 引导的联合 UCT / MCTS

基于 COMP8620 Assignment 2、课程课件与 lrlorry/8620 的 GitHub 实现

版本 5c7e9c20 ·2026 年 10 月 25 日·研究生教学详解

# 目录

**第一讲：从机器人随机运动的数学本质出发，理解 B1 —MDP Modelling（3 分） 2**
先谈 Claude 所说的“理论上限” . . . 3
整个 Part B 的知识地图 . . . 3
1. 首先，机器人到底在解决什么? . . . 3
2. 为什么用 MDP? . . . 4
3. State：为什么状态不仅包含坐标? . . . 4
4. Action：为什么是联合动作? . . . 6
5. Transition：这是 B1 的数学核心 . . . 6
6. Reward：规划器究竟在优化什么? . . . 8
7. 最终目标：不是走最短路，而是最大化期望累计奖励 . . . 9
第一讲总结：你要建立的思维模型 . . . 9

**第二讲：从 Bellman Equation 到 MCTS 11**
一、Bellman 方程：为什么不能只考虑下一步? . . . 11
二、Bellman 方程已经有了，为什么还需要 MCTS? . . . 12
三、MCTS 的本质：通过模拟，学习哪些动作值得继续探索 . . . 14
四、第一阶段：Selection —怎么决定继续探索哪个动作? . . . 15
五、第二阶段：Expansion —为什么不能展开所有联合动作? . . . 16
六、第三阶段：Simulation —为什么还需要 Rollout? . . . 18
七、随机结果：为什么同一个动作还会有多个子节点? . . . 20
八、第四阶段：Backpropagation —模拟结果怎样更新 Q? . . . 21
九、整套 MCTS 在 `_search()` 中怎样串起来? . . . 22
十、最容易忽视的地方：搜索结束后，为什么不直接执行 Q 最大的动作? . . . 22
十一、最后：Online Planner 怎么控制时间? . . . 23
第二讲总结 . . . 24

**第三讲：真正读懂新版 B2——单机有限时域 Value Iteration 如何引导联合 MCTS 26**
一、先用一张结构图建立宏观认识 . . . 26
二、为什么 BFS / Dijkstra 不够? . . . 26
三、先把老师的随机移动规律写准确 . . . 27
四、有限时域 VI：时间不是可有可无的下标 . . . 28
五、从 Bellman 公式到 `scipy.sparse`：源码逐段拆解 . . . 29
六、`_value_options()`：怎样把最大化 Q 变成最小化动作成本? . . . 30
七、联合动作生成：VI 本身不懂其他无人机，怎么办? . . . 31
八、`_leaf()`：这次也升级成 VI Value，而不只是动作提案 . . . 32
九、为什么这次 VI 升级比「换一种启发式」更深? . . . 32
十、课堂演练：你能推导这三个问题吗? . . . 33

**第四讲：B3——从 `plan()` 到 `step()` 的 Python 闭环系统（8 分） 34**
一、首先区分三种程序角色 . . . 34
二、初始化时的 VI 与每步在线 MCTS 是两项不同的计算 . . . 34
三、一次 Episode 在代码中怎么走? . . . 34
四、为什么代码中的 `simulate()` 还要知道剩余时间? . . . 35
五、10 秒规则不是一句口号，而是一个接口契约 . . . 35
六、为何每个真实时间步都重建搜索根节点? . . . 36
七、从日志读取真实算法行为 . . . 36
八、`--visualize` 与科研图表分别能看到什么? . . . 37
九、B3 的正确性还依赖哪些边界条件? . . . 37
最新版发布验证：哪些数据应该记住? . . . 37
第四讲总结 . . . 37

**第五讲：B4（20 分）——怎样用实验真正检验新版 VI-MCTS? 38**
一、先分清实验中的四种“次数” . . . 38
二、S1 全因子：为什么是 $3^3 \times 20 = 540$? . . . 38
三、S1 仍然有两类混杂：为什么还需要 S2? . . . 39
四、S3 消融：不是“关掉一个超参数”这么简单 . . . 40
五、S4 搜索预算：为什么更大的计算预算不保证更好的表现? . . . 40

六、S5 随机生成：扩大覆盖，但不能混淆变量 . . . . . 41
七、统计部分：成功率、完成步数、配对差与 CI 不一样 . . . . . 41
八、两套规划预算——为什么不能混为一谈? . . . . . 42
九、如何把 B4 写成有说服力的研究故事? . . . . . 42

**第六讲：读懂新版 B4 的真实结果——VI 有效在哪里，MCTS 为什么未必更好?** **43**
一、S1：三个复杂度因素，哪个最重要? . . . . . 43
二、S2：控制实验告诉我们「为什么」，不只是「是什么」 . . . . . 44
三、S3 消融：哪几个组件真正带来了收益? . . . . . 44
四、最关键的正式比较：旧 Dijkstra A 对新版 VI B . . . . . 45
五、为什么完整版 MCTS 有时反而不如 VI-Greedy? . . . . . 46
六、S4：为什么多试验 2,000 次也不总是更好? . . . . . 47
七、S5 与压力场景：哪些数据发生了真实变化? . . . . . 47
八、把本讲归结为三个有层次的结论 . . . . . 48

**第七讲：理论成功率上限、狭窄通道与“算法已经最优”的逻辑边界** **49**
一、成功率最大化与折扣奖励最大化，不是同一目标 . . . . . 49
二、有限时域成功率 Bellman Backup . . . . . 49
三、墙壁缺口：理论最优约 70.5%，当前算法 14/20 . . . . . 50
四、共享走廊：理论最优约 63.1%，当前算法 13/20 . . . . . 50
五、四机高噪声：为什么不能计算完整联合理论上限? . . . . . 50
六、一个容易考错的概率推论：单机成功率高，联合成功率不一定高 . . . . . 51
七、为什么风险公式会造成“方向不敏感”的现象? . . . . . 51
八、三类失败机制必须分开讨论 . . . . . 51
九、什么叫“证据有力，但还不构成证明”? . . . . . 51
十、局限性与后续研究方向：可以提，不要假装已经做过 . . . . . 52

**第八讲：像机器人规划研究者一样解释项目——代码地图、口试问答与证明边界** **53**
一、把整个系统浓缩成一张因果结构图 . . . . . 53
二、代码阅读地图：从上到下应该怎么看? . . . . . 53
三、最重要的 20 个口试问题（附参考回答） . . . . . 54
四、给考官的 90 秒英文技术介绍（示范文本） . . . . . 56
五、必须能背后理解而非机械背诵的公式索引 . . . . . 56
六、十道真正考察理解的练习题（附要点） . . . . . 56
七、三层理解标准 . . . . . 57

**附录 A：原题要求、课程出处与项目版本** **58**

**附录 B：逐项学习检查清单** **58**

> **更新说明（与旧版的关系）**：本讲义直接继承此前 60 页版本的八讲结构、前两讲的完整推导、教学例子和讲解风格。涉及新版算法的内容进行了必要修订；第三讲和第五至八讲根据提交的 VI-guided MCTS、最新报告和实验记录系统重写。算法来源、教学算例与统计证据明确区分；不把旧版 Dijkstra 数据当作新 VI 实测。

**阅读路线**：第一讲 B1（是什么）→ 第二讲 B2 课堂理论（为什么）→ 第三讲单机 VI 与联合引导（怎么做）→ 第四讲代码执行（怎么跑）→ 第五讲研究设计（怎么检验）→ 第六讲实验分析（得出了什么）→ 第七讲精确成功率上限与局限性（能证明什么）→ 第八讲口试与公式索引（能否解释给别人）。

# 第一讲：从机器人随机运动的数学本质出发，理解 B1 —MDP Modelling（3 分）

对应 Week 5《MDP Solving》、作业 a2.pdf Part B Q1、报告 A2/partB.tex 与仿真器 A2_code/multi_drone.py。

我们从最基础的思想开始，不急着讲 MCTS 的四个阶段。

我希望你最终不仅知道代码在做什么，还能回答三个更深的问题：

1. **为什么这个机器人问题需要 MDP，而不只是 A* 或 Dijkstra?**

2. **为什么已经知道运动概率，仍然很难计算最优动作?**
3. **为什么你的系统需要 MCTS、单机 Value Iteration 引导和 Monte Carlo Simulation 一起工作?**

我重新核对了你的 GitHub 当前代码、A2/partB.tex 报告和老师的 Week 5–6 课件。我们按照报告的 B1 → B2 → B3 → B4 顺序学习，每次只深入一部分。

## 先谈 Claude 所说的“理论上限”

Claude 的判断有合理的物理直觉：**你的任务中存在无法通过规划算法完全消除的运动随机性**。

例如，某架无人机即使选择了正确方向，也可能因为随机偏移撞上障碍物。对于一个必须经过狭窄通道的任务，这种风险可能无法完全避免。

但要严格区分:

- **不可避免的随机风险：**可以由运动模型支持。
- **当前算法已经达到理论最优成功率：**需要额外的数学证明。

设 $\pi$ 是一个规划策略，任务成功率的理论最优值为:

$$\boxed{P_{\max} = \sup_{\pi} P_{\pi}(\text{所有无人机成功到达目标})}$$

要证明你的算法达到这个上限，需要计算或严格约束 $P_{\max}$，再与算法成功率比较。

最新版的墙壁缺口 14/20、四机高噪声 6/10 等实验数据，不能单独证明已经达到理论上限。它们只能说明算法在这些随机环境中的实际表现。

另外还有一个特别重要的区别：**你的 MDP 优化目标是期望折扣奖励，不是单纯最大化成功率**。有时候更快但略有风险的策略，可能具有更高的期望奖励。

这个区别是理解整份作业的关键。

## 整个 Part B 的知识地图




图 1: Part B 知识地图

从智能机器人领域的角度，Part B 贯穿三个层次:

**第一层：建模（Modelling）**。把现实任务翻译成可以计算的数学问题。

**第二层：决策（Planning）**。在运动结果不确定的情况下，寻找期望回报较好的动作。

**第三层：验证（Evaluation）**。通过实际随机实验，判断算法是否真的有效。

今天先把第一层讲透。

## 1. 首先，机器人到底在解决什么?

想象你控制四架无人机。

每架无人机都有自己的终点，环境中存在障碍物。你需要同时决定四架无人机下一步怎么走。

如果运动完全确定，问题比较直观：找到一组不发生碰撞的路径，然后执行即可。

但老师给定了一个额外条件:

**你下达的动作，不一定是无人机最终执行的运动方向**。

例如，你命令无人机向右移动：




图 2: 预期移动与随机偏移

绿色为想执行的方向；红色为运动扰动后可能发生的其他移动。图仅用于说明随机动作效果。

这意味着规划器不能只回答：

从当前位置到目标，最短路径是哪一条?

而要回答：

我现在采取哪个动作，考虑随机运动、碰撞和后续决策后，长期期望收益最好?

这就是 **Sequential Decision Making under Uncertainty（不确定性下的序贯决策）**。

## 2. 为什么用 MDP?

MDP 是 Markov Decision Process，马尔可夫决策过程。

你的报告将它写成：

$$\boxed{\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)}$$

不要急着记五个符号。

它们实际上对应五个非常自然的问题：

| 数学对象 | 要回答的问题 |
|---|---|
| $S$: State | 机器人当前处于什么情况? |
| $A$: Action | 机器人现在可以做什么? |
| $T$: Transition | 做出动作后，可能发生什么？概率是多少? |
| $R$: Reward | 发生这样的结果，好还是不好? |
| $\gamma$: Discount Factor | 如何权衡现在与未来的收益? |

**MDP 的本质，就是用这五个对象，把随机环境下的连续决策过程数学化。**

还有一个必须掌握的性质：

$$\boxed{P(s_{t+1} \mid s_{0:t}, a_{0:t}) = P(s_{t+1} \mid s_t, a_t)}$$

这叫 **Markov Property（马尔可夫性质）**。

意思是：如果当前状态定义得足够完整，那么在预测下一状态时，不需要重新查看全部历史。

现在我们检查你的 B1 状态是否满足这个要求。

## 3. State：为什么状态不仅包含坐标?

报告写的是：

$$\boxed{s_t = ((p_i, r_i)_{i=1}^{N}, t)}$$

逐个解释。

### $p_i$：第 $i$ 架无人机的位置

$$p_i = (x_i, y_i, z_i)$$

例如，两个无人机分别位于：

$$p_1 = (2, 3, 1), \qquad p_2 = (5, 2, 0)$$

由于需要集中式规划，必须同时知道两个位置。

### $r_i$：是否已经到达目标

$$r_i \in \{0, 1\}$$

- $r_i = 0$：还没有到达目标。
- $r_i = 1$：已经到达目标。

为什么需要这个变量?

因为老师提供的仿真器有一条非常重要的规则：

**无人机到达目标后，会永久停留在那里，并继续占据那个格子。**

看实际代码：

```
targets[reached] = prev_pos[reached]
```

这就是你之前共享走廊问题产生复杂协调需求的根本原因。

并且，无人机第一次到达目标时才获得目标奖励。因此 reached 同时影响未来运动和奖励计算。

### $t$：当前时间步

为什么还需要 $t$?

因为老师规定了最大执行步数 $H$。

假设两次出现完全相同的无人机位置：

- 第一次只执行了 5 步，还有 75 步。
- 第二次已经执行了 79 步，只剩 1 步。

虽然位置相同，但未来能够做出的决策数量不同。

所以严格的有限时域 MDP 必须能够区分这两种情况。

在实际实现中：

```
remaining = max(
    0,
    int(self.cfg.max_num_steps) - self.steps
)
```

报告在数学状态中明确写了 $t$，而代码使用 self.steps 在规划器外部维护时间信息。

**这里体现了一种重要的工程原则：数学状态的定义与程序内部的数据表示不必完全相同，但必须保留决策所需的信息。**

### 为什么报告还定义了吸收状态 $\perp$?

报告写道：

$$T(\perp, a, \perp) = 1$$

它表示任务一旦终止，就不会再回到正常状态。

可能的终止原因包括成功、碰撞和达到时间限制。

在代码中，终止由 done 和真实执行的时间限制表示；报告用 $\perp$ 将它们统一为数学上的吸收状态。

## 4. Action：为什么是联合动作?

对于三维运动，每架无人机的位移为:

$$(\Delta x, \Delta y, \Delta z) \in \{-1, 0, 1\}^3$$

排除原地不动:

$$3^3 - 1 = 26$$

因此 $N$ 架无人机的联合动作:

$$\boxed{a = (a_1, \dots, a_N)}$$

联合动作空间的大小是:

$$\boxed{|A| = 26^N}$$

四架时达到:

$$|A| = 456{,}976$$

这引出了整个 Part B 最重要的计算困难:

**单架无人机的动作不多，但集中式联合决策会让动作空间随无人机数量指数增长。**

而且，仅仅计算出四架无人机各自的最短路径还不够，因为它们的动作会相互影响。

### 对应代码

在 planner.py:

```
def encode_action(digits, base):
    value, multiplier = 0, 1
    for digit in digits:
        value += int(digit) * multiplier
        multiplier *= base
    return value
```

其数学形式为:

$$a_{\text{int}} = \sum_{i=1}^{N} a_i B^{i-1}$$

这只是联合动作的编码方式，不是规划算法本身。

## 5. Transition：这是 B1 的数学核心

现在来到最重要的部分。

你的报告写了:

$$\boxed{T(s,a,s') = \sum_{y:F(s,y)=s'} \prod_{i=1}^{N} q_i(y_i \mid p_i, r_i, a_i)}$$

看起来复杂，但它实际上只有两步。

**第一步：分别产生每架无人机的随机移动**

仿真器中:

```
d = self.dist_field[
    prev_pos[:, 0],
    prev_pos[:, 1],
    prev_pos[:, 2]
]

p_succ = d / (d + self.cfg.alpha)
```

即:

$$p_{\text{succ}} = \frac{d}{d+\alpha}$$

其中 $d$ 是当前位置的障碍物距离场数值，$\alpha$ 控制运动的不确定程度。

假设:

$$d = 2, \qquad \alpha = 0.5$$

那么:

$$p_{\text{succ}} = \frac{2}{2.5} = 0.8$$

在所有方向都不越界的情况下，预期动作被赋予 0.8 的概率，其余方向分享剩下的 0.2。

注意，代码还会排除越界方向并重新归一化，因此靠近边界时最终概率可能不同。

对于第 $i$ 架无人机，这个概率分布记作:

$$q_i(y_i \mid p_i, r_i, a_i)$$

报告中的乘积:

$$\prod_{i=1}^{N} q_i(y_i \mid p_i, r_i, a_i)$$

表示给定当前状态和联合动作，各无人机的原始运动结果按照模型条件独立采样。

**第二步：统一处理碰撞和终止**

这一步用函数 $F(s,y)$ 表示。

仿真器会检查:

```
obstacles_hit = self.obstacles[
    targets[:, 0],
    targets[:, 1],
```

```
    targets[:, 2]
]
```

然后检查无人机之间的碰撞：

```
_, counts = np.unique(
    next_pos,
    axis=0,
    return_counts=True
)

veh_collisions = int(counts[counts > 1].sum())
```

这里有一个非常深刻的区别：

**每架无人机的随机运动可以条件独立，但联合任务的结果并不独立。**

例如，两架无人机分别以很高概率到达同一个格子。

从各自运动角度看，两次移动都可能执行成功；但从联合任务角度看，它们发生了碰撞。

因此必须在独立采样之后，再统一判断整个系统的下一状态和任务后果。

这就是 B1 转移公式中：

$$\underbrace{\prod_i q_i}_{\text{各无人机随机运动}} \quad \longrightarrow \quad \underbrace{F(s,y)}_{\text{全局碰撞与状态更新}}$$

的真正含义。

它也回答了一个问题：**为什么这里必须是 Centralised Planning，而不是把几个单无人机规划器简单并排运行?**

因为联合动作的价值受到其他无人机的状态和行动影响。

## 6. Reward：规划器究竟在优化什么?

这是另一个特别重要的知识点。

你的报告给出的采样奖励是：

$$\boxed{\tilde{r} = Nc_{\text{step}} + C(K_O + K_V) + G\sum_i \Delta r_i}$$

可以理解为三部分：

$Nc$：**运动时间成本**。每执行一步都付出负奖励，鼓励尽快完成任务。

$C(K_O + K_V)$：**碰撞惩罚**。撞到障碍物或其他无人机，产生较大的负奖励。

$G\sum_i \Delta r_i$：**首次抵达目标的奖励**。鼓励每架无人机完成自己的任务。

例如，在一个两架无人机、步成本为 $-1$、碰撞惩罚为 $-100$、目标奖励为 100 的配置中：

| 实际发生的事件 | 本步奖励 |
|---|---|
| 两架都还在正常移动 | $-2$ |
| 一架首次到达目标，无碰撞 | 98 |
| 一架撞到障碍物，无新目标到达 | $-102$ |

这里的奖励是一**次随机执行后实际得到的数值**：

$$\tilde{r}(s, a, y)$$

而数学上的期望奖励是：

$$R(s, a) = \mathbb{E}[\tilde{r}(s, a, y) \mid s, a]$$

区别很重要：前者是一次模拟结果，后者要对所有可能结果求概率加权平均。

MCTS 正是通过许多次随机模拟，逐渐估计动作的长期价值。

## 7. 最终目标：不是走最短路，而是最大化期望累计奖励

报告给出：

$$\max_{\mu} \mathbb{E}_{\mu}\left[\sum_{t=0}^{\tau-1} \gamma^t \tilde{r}_t\right]$$

这里：

- $\mu$：决策策略，规定每个状态应该选择什么动作。
- $\gamma$：折扣因子。
- $\tau$：任务实际终止时刻。
- $\tilde{r}_t$：第 $t$ 步获得的奖励。

从研究生层面的理解来看，重点在于：

**一条路径的价值，不只是几何长度，而是执行这条策略时所有可能随机结果的期望累计收益。**

例如，一条路径很短，但会多次贴着障碍物行进；另一条稍长，但大部分时间远离障碍物。MDP 会通过转移概率、碰撞惩罚和未来奖励综合权衡这两种选择。

这也是为什么后面 B2 的规划器需要估计：

$$Q(s, a)$$

即**当前状态下选择动作 $a$，随后继续决策，能够获得多大期望累计回报。**

MCTS 要解决的问题，正是如何在有限计算预算下近似比较这些 $Q(s, a)$。

## 第一讲总结：你要建立的思维模型

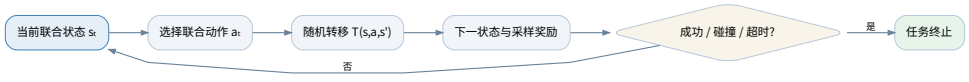



图 3: MDP 闭环流程

你现在需要把三件事牢牢记住：

**第一，MDP 是问题的数学模型，不是具体的求解算法。**

**第二，运动随机性由 $T$ 描述，目标由 $R$ 描述。它们决定了什么策略是好的。**

**第三，已知 $T$ 和 $R$，不代表最优策略已经知道。**因为要考虑未来许多步、多个无人机及所有可能的随机后果，求解仍然很困难。

对应到程序，最关键的一行是：

```
successor, reward, done, info = execution.step(action)
```

它就是整个 MDP 一次真实状态转移的工程实现。

而 planner.py 中的:

```
return self.env.simulate(state, action)
```

则允许规划器在真正行动之前，提前抽样研究不同动作可能产生的未来。

**这两行代码之间的区别，正是下一讲 B2：Online MDP Solving 与 MCTS 的起点。**

### 一个检验你是否真正理解 B1 的问题

假设有两架无人机，每架已经分别找到了自己的最短路径，并且两条路径在几何上没有共享格子。

**为什么在这个作业中，我们仍然不能保证它们不会发生碰撞?**

关键不在路径规划本身，而在你刚学到的转移模型 $T$。如果能解释清楚这个问题，就说明你已经抓住了 B1 的核心。新版 VI 更进一步把已知随机转移直接用于单机 Bellman 计算，但不会改变 B1 的联合随机模型。

下一讲我们将从 **Bellman Equation → 为什么精确求解困难 → MCTS 如何近似求解 → _search() 逐行分析** 这条主线进入 B2。

# 第二讲：从 Bellman Equation 到 MCTS

对应 Week 5《MDP Solving》、Week 6《MDP Solving Continued》、A2 Part B Q2，以及你 GitHub 中的 planner.py。

第一讲我们建立了 MDP:

$$\mathcal{M} = (S, A, T, R, \gamma)$$

现在我们知道了无人机的状态、可执行的动作、运动发生随机偏移的概率，以及完成任务和发生碰撞的奖励。

但是，这些知识还没有告诉我们:

**在当前状态下，究竟应该选择哪一个联合动作?**

这就是第二讲要解决的核心问题。

我希望你理解一个贯穿本讲的思想:

> **MDP 告诉我们什么是好的决策；Bellman 方程刻画最优决策；MCTS 则试图在有限计算时间内近似找到这样的决策。**

## 一、Bellman 方程：为什么不能只考虑下一步?

先从一个简单问题开始。

假设无人机有两种选择:

- 动作 A：下一步就靠近目标，但未来必须经过障碍物密集区域。
- 动作 B：下一步看起来绕远了，但之后可以通过更安全的区域。

如果使用纯粹的贪心策略（Greedy），可能会选择 A。

但我们真正关心的并不是下一步距离减少了多少，而是**整个任务的期望累计奖励**。

### 1. 定义状态价值 $V(s)$

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\tau-1} \gamma^k r_{t+k} \,\middle|\, s_t = s\right]$$

它表示:

从状态 $s$ 出发，按照策略 $\pi$ 不断采取动作，未来预计能够获得多少累计折扣奖励。

注意，$V(s)$ 衡量的是**状态的价值**。

### 2. 定义动作价值 $Q(s, a)$

$$Q^{\pi}(s, a) = \mathbb{E}\left[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s, a_t = a\right]$$

它表示:

在状态 $s$ 下，首先执行动作 $a$，然后按照策略 $\pi$ 继续行动，预计能获得多少奖励。

比较两个公式:

$$\underbrace{V(s)}_{\text{这个状态有多好}} \qquad \underbrace{Q(s, a)}_{\text{现在选择这个动作有多好}}$$

**你的 MCTS 主要就是在估计不同候选动作的 $Q(s, a)$。**

### 3. Bellman Optimality Equation

对于还剩 $h$ 步的有限时域问题:

$$\boxed{Q_h^*(s,a) = \mathbb{E}\left[r + \gamma V_{h-1}^*(s') \mid s, a\right]}$$

$$\boxed{V_h^*(s) = \max_{a \in A(s)} Q_h^*(s,a)}$$

其中终止状态的未来价值为零，$V_0^*(s) = 0$。

这两个公式合起来就是最优性递推。

理解第一条公式:

$$\underbrace{Q_h^*(s,a)}_{\text{选择动作}a\text{ 的长期价值}} = \underbrace{\mathbb{E}[r]}_{\text{当前奖励}} + \underbrace{\gamma \mathbb{E}[V_{h-1}^*(s')]}_{\text{未来价值}}$$

例如，当前选择一个动作，它可能带来:

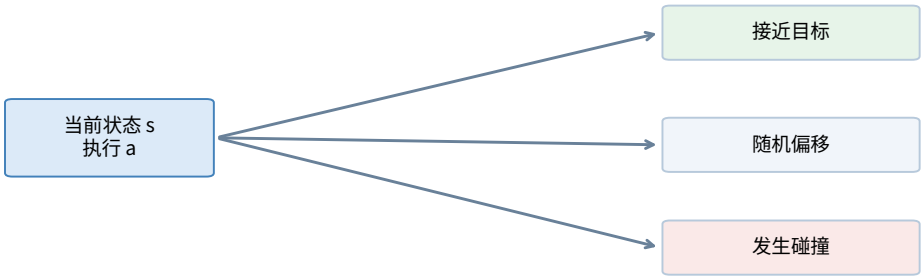



图 4: 同一动作的随机结果

同一个动作可能产生多个随机结果，各自具有不同的奖励及后续价值。

因此你必须把这些可能结果按照它们的概率综合考虑。

这就是 Bellman 方程中的期望算子 $\mathbb{E}$。

这里有一个研究生阶段非常重要的认识:

**最优动作不一定具有最高的立即奖励，也不一定使几何距离下降最多，而是具有最高的期望长期价值。**

你的报告中，优化目标也是期望折扣回报，而非直接最大化成功率。

## 二、Bellman 方程已经有了，为什么还需要 MCTS?

这可能是整份 Part B 最值得理解的问题。

如果我们已经知道最优性方程，似乎只需要把它计算出来就行了。

问题在于：**计算规模可能过于庞大**。

### 1. 动作数量增长

对于四架三维无人机:

$$|A| = 26^4 = 456{,}976$$

一个状态就可能对应几十万个联合动作。

### 2. 每个动作还对应多个随机后继

例如，某个联合动作 $a$:

$$(s,a) \longrightarrow \begin{cases} s'_1 \\ s'_2 \\ s'_3 \\ \vdots \end{cases}$$

这些后继来自运动扰动的不同组合。

所以求解一个 $Q(s,a)$，需要考虑多个可能的 $s'$。

而每个 $s'$ 又需要考虑下一步动作。

再往后，又会有新的随机后继。

这种嵌套递推使完整求解的计算量很大。

### 3. Offline 与 Online Planning 的本质区别

Week 5 第 20 页讨论了 Computing the Policy Online。

| Offline Planning | Online Planning |
|---|---|
| 尝试预先计算大量状态的策略或价值 | 集中计算当前真实状态下的动作 |
| 更强调提前求得完整策略 | 计算与实际执行交替进行 |
| 可能需要处理大量无关状态 | 重点搜索当前状态附近的未来 |
| 计算规模大时比较困难 | 可以在有限预算后给出近似决策 |

对于你的作业，正确的工作流程是:

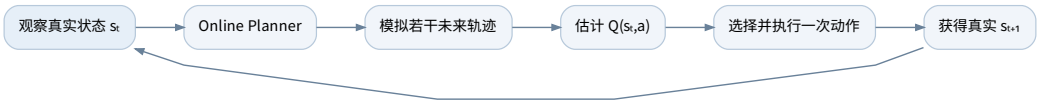



图 5: Online Planner 与真实执行交替

这里最关键的区别是:

**规划器可以模拟很多次未来，但真实环境每次只执行一个动作。**

对应你的代码:

```
# 在规划器内部进行假想的未来模拟
successor, reward, done, _ = self.env.simulate(state, action)

# 真正让无人机执行选定的动作
successor, reward, done, info = execution.step(action)
```

前者在 planner.py；后者在 run_u8076655.py。

这就是为什么 MCTS 特别适合老师提供的 MultiDroneUnc：它直接提供了一个能够采样随机后继的 **Generative Model（生成式模型）**。

你不必提前构建完整的 $T(s,a,s')$ 矩阵。

## 三、MCTS 的本质：通过模拟，学习哪些动作值得继续探索

MCTS 全称:

**Monte Carlo Tree Search（蒙特卡洛树搜索）**

它结合两个思想。

**Monte Carlo：**用随机采样来估计难以精确计算的期望。

**Tree Search：**将计算集中在目前看来有价值的决策分支上。

因此 MCTS 可以理解为:

| 通过有限次随机模拟，逐渐改进当前动作的价值估计 |
|---|

注意，MCTS 不是强化学习中的 DQN，也不需要训练神经网络。

它是在当前决策时刻使用模型进行规划。

### 为什么是树?

假设无人机当前处于状态 $s_0$，有两个候选动作 $a_1, a_2$。

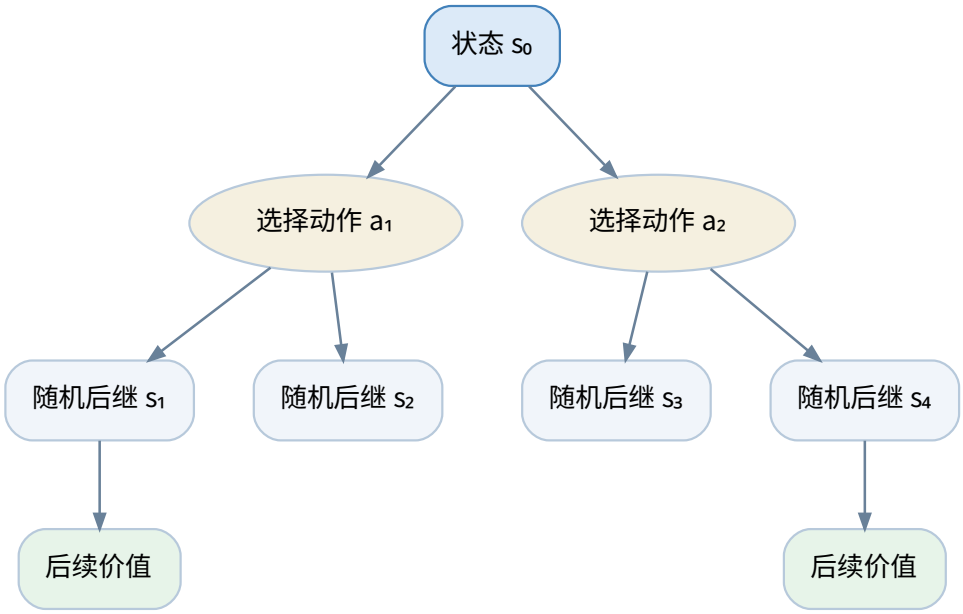



图 6: 带有动作与随机后继的搜索树

必须注意，这里有两种不同的分支:

**动作分支：**规划器可以选择哪个动作?

**随机结果分支：**某个动作执行后，环境实际产生哪个后继?

Week 5 第 31 页将 MDP 的 MCTS 搜索树描述为 AND–OR Tree。

它表达的就是这种“选择动作”与“考虑随机结果”的交替结构。

你代码中的:

`edge.children`

正是用来保存同一个动作对应的不同随机后继状态。

### MCTS 的四阶段

Week 5 第 27–30 页介绍了四个组成部分：

| 阶段 | 解决的问题 |
|---|---|
| Selection | 已有候选动作中，继续模拟哪个? |
| Expansion | 什么时候允许新动作或新状态进入树? |
| Simulation / Rollout | 搜索树尚未展开的未来，怎么估计? |
| Backpropagation | 模拟得到的回报，怎样更新之前的决策价值? |

接下来我们真正进入你的 planner.py，看这四个阶段如何组成一个可以运行的在线求解器。

## 四、第一阶段：Selection —怎么决定继续探索哪个动作?

先看你的代码：

```
unseen = [
    a for a, edge in node.actions.items()
    if edge.visits == 0
]

if unseen:
    action = unseen[0]
else:
    log_n = math.log(node.visits + 1)
    action = max(
        node.actions,
        key=lambda a:
            node.actions[a].value
            + self.exploration *
              math.sqrt(log_n / node.actions[a].visits)
    )
```

对应 planner.py 中的 _search()。

这段代码分成两种情况。

**情况一：存在还没有模拟过的候选动作。**

优先模拟它一次。

因为如果一个动作从未被尝试，我们根本没有它的价值估计。

**情况二：已有候选动作都至少模拟了一次。**

使用 UCT：

$$a^* = \arg\max_a \left[ Q(s,a) + c_{\text{UCT}} \sqrt{\frac{\ln(N(s)+1)}{N(s,a)}} \right]$$

当前代码：

$$c_{\text{UCT}} = 0.35$$

这就是 Week 5 第 33 页讲的 UCB/UCT。

### 为什么不直接选择 Q 最大的动作?

假设有两个动作:

| 动作 | 当前平均回报 $Q$ | 模拟次数 |
|---|---|---|
| A | 0.18 | 100 |
| B | 0.14 | 10 |

只看平均回报，会选择 A。

但是 B 只模拟了 10 次，我们可能尚未充分了解它。

假设当前节点总访问次数为 110，那么:

$$\mathrm{UCT}(A) = 0.18 + 0.35\sqrt{\frac{\ln 111}{100}} \approx 0.256$$

$$\mathrm{UCT}(B) = 0.14 + 0.35\sqrt{\frac{\ln 111}{10}} \approx 0.380$$

所以 MCTS 此时反而会选择 B。

原因是:

**B 的平均收益虽然暂时较低，但它被探索得还不充分。**

这叫 Exploration–Exploitation Trade-off。

### 交互理解 UCT 探索系数（PDF 静态版）

| 探索系数 $c$ | UCT(A) | UCT(B) | 此时优先探索 |
|---|---|---|---|
| 0 | 0.180 | 0.140 | A |
| 0.20 | 0.223 | 0.277 | B |
| 0.35 | 0.256 | 0.380 | B |
| 0.80 | 0.354 | 0.689 | B |

原对话中的可拖动 UCT 探索系数滑块在 PDF 中替换为同一模型的静态数值表。这些数字是教学示例，不是代码的真实运行数据。

**注意：UCT 是搜索时使用的动作选择公式，并不等于最终真实执行时的动作选择规则。**

你的代码在搜索完成后，还有一个独立的 Root Decision 机制。后面会讲。

## 五、第二阶段：Expansion —为什么不能展开所有联合动作?

这里是你实现与最基本 MCTS 的一个重要区别。

理论上，四架三维无人机最多有:

$$26^4 = 456{,}976$$

个联合动作。

假设把所有动作都加入搜索树，然后对每个动作平均分配模拟次数。

在单步最多 2,000 次搜索试验的预算下，连每个动作模拟一次都做不到。

因此必须回答:

**哪些动作值得进入搜索树? 应该允许多少个动作进入?**

你的方法叫 **Progressive Widening（渐进拓宽）**。

### 1. 核心公式

代码:

```
width = max(
    1,
    math.ceil(
        self.widening * math.sqrt(node.visits + 1)
    )
)
```

当前:

$$\texttt{widening} = 0.5$$

因此:

$$\boxed{K(s) = \left\lceil 0.5\sqrt{N(s)+1} \right\rceil}$$

同时还受可提案动作数量的限制。

我们来看几个数:

| 节点访问次数 $N(s)$ | 最多允许的候选动作数 |
|---:|---:|
| 0 | 1 |
| 100 | 6 |
| 500 | 12 |
| 2,000 | 23 |

发现没有?

即使原始动作空间中有几十万个动作，搜索树在访问 2,000 次的量级下，也只允许加入几十个候选动作。

这就大大提高了有限计算预算的利用效率。

但它也引入一个代价：**如果真正好的动作没有被加入候选集合，UCT 根本无从发现它**。

### 2. 那么谁决定加入哪些动作?

对应代码:

```
action = self._proposal(
    state,
    randomize=bool(node.actions),
    remaining=remaining
)
```

_proposal() 是你算法里非常关键的启发式动作生成函数。

它不是纯随机生成联合动作，而是使用**单机 VI 的有限时域动作价值**来排序；还叠加机间风险与目标占用约束:

Single-drone finite-horizon VI + 机间风险和预约目标格 + 联合随机 MCTS 搜索

具体而言，最新版主要依据 $Q_{i,h}(p,a)$ 对每架无人机的合法候选动作排序，再考虑其他无人机占用目标格子以及机间随机偏移风险。旧 Dijkstra 是上一版指导方案，只有超出 VI 预计算规模时才作为回退。

第一次加入动作时，使用 Greedy Proposal；之后在有偏随机采样和探索采样之间选择。

所以你的 Expansion 实际上有两层:

$$\underbrace{\text{Progressive Widening}}_{\text{决定加入多少动作}} + \underbrace{\text{Heuristic Proposal}}_{\text{决定优先加入哪些动作}}$$

这是整套系统最重要的工程设计之一。

这里要区分课程依据:

Week 5 明确讲了 MCTS Expansion；Progressive Widening 和这套风险感知动作提案，是你针对本题进行的具体设计扩展，而不是课件直接给定的实现。

## 六、第三阶段：Simulation —为什么还需要 Rollout?

这是我希望你特别吃透的部分。

MCTS 的搜索树不可能无限往未来展开。

假设当前无人机距离终点还要 20 步，而你的搜索树只探索了 5 步。

那么，搜索到第 5 步时，怎么判断这个状态好不好?

一种极端做法是:

$$V(s) = 0$$

也就是把尚未模拟的未来价值视为零。

但这可能产生严重偏差。

因为一个已经非常接近目标的状态，和一个仍然远离目标的状态，显然不应该具有相同价值。

### 1. Rollout 的作用

Rollout 的思想是:

**从新发现的状态出发，再按照一个比较简单的策略模拟未来几步，以此估计状态价值。**

Week 5 第 34 页明确建议可以使用确定性问题的 Greedy 解作为启发式 Rollout。

你的代码:

```
for _ in range(min(self.rollout_depth, remaining)):
    explore = self.rng.random() < self.rollout_epsilon

    action = self._proposal(
        state,
        randomize=False,
        explore=explore,
        remaining=remaining - len(rewards)
    )

    state, reward, done, _ = self._simulate(
        state, action
    )
```

默认参数:

$$H_{\text{rollout}} = 8$$

$$\epsilon_{\text{rollout}} = 0.10$$

表示 Rollout 最多额外模拟 8 步，每一步以 10% 的概率使用探索提案，其余情况使用目标导向启发式。

但这 10% 是**规划策略的探索概率**，与真实无人机的运动扰动概率不是一回事。

### 2. 为什么还需要 Leaf Value?

如果 Rollout 的 8 步走完了，仍然没有抵达目标怎么办?

你的代码:

```
tail = (
    0.
    if terminal
    else self._leaf(state, remaining - len(rewards))
)
```

即:

- 已经终止：未来价值为 0。
- 还没有终止：调用 _leaf() 估计剩余价值。

**最新版 Leaf Value 在 VI 预计算成功时主要使用每架无人机的 $V_{i,h}$**，并对冻结无人机的剩余步成本与已冻结目标切断路径的情况作近似处理。BFS 最短距离保留用于连通性检查及 VI 不可用时的回退估值。

注意这个分工:

| 组件 | 使用什么信息 |
|---|---|
| _value_iteration() | 对每架无人机单独做有限时域随机 Bellman Backup |
| _value_options() | 用 $Q_{i,h}$ 决定哪些动作优先被提案 |
| _distance_map() | BFS 距离，用于静态连通性检查与回退 |
| _cost_map() | 旧版 Dijkstra 指导，以及 VI 超出规模阈值时的回退 |
| _leaf() | 优先使用 $V_{i,h}$，否则使用 BFS 近似估值 |

**VI 的单机 Bellman Value 没有代替联合 MDP 的真实价值函数。**它在联合搜索中起到相对准确但忽略他机耦合的先验作用；MCTS 依然通过真实联合随机生成模型估计动作长期回报。

第三讲将从公式到源码完整推导这个新机制。

### 3. Rollout 到底在估计什么?

假设它模拟三步，得到奖励:

$$r_0, r_1, r_2$$

最终的近似回报是:

$$\boxed{\widehat{G} = \frac{r_0}{\kappa} + \gamma \frac{r_1}{\kappa} + \gamma^2 \frac{r_2}{\kappa} + \gamma^3 \frac{\widehat{V}(s_3)}{\kappa}}$$

其中 $\kappa$ 是奖励归一化系数。

代码中 Rollout 使用了倒序折扣累加:

```
for reward in reversed(rewards):
    tail = reward + self.gamma * tail
```

注意，此处 rewards 已经除以 $\kappa$，而 _leaf() 返回值也已经归一化。

**为什么说 Rollout 可能影响算法性能?**

因为我们真正想要的是最优价值:

$$V^*(s)$$

但 Rollout 通常只是按照一个启发式策略执行，所以它给出的只是近似价值:

$$\widehat{V}(s)$$

如果启发式策略本身对某些情况判断错误，那么 MCTS 也可能继承这种偏差。

这正是你墙壁缺口场景中出现反复绕行的可能原因之一，但仅凭轨迹不能断定它是唯一原因。

## 七、随机结果：为什么同一个动作还会有多个子节点?

看 _search() 里的代码:

```
successor, reward, done, _ = self._simulate(
    state, action
)

key = np.asarray(
    successor, dtype=np.int32
).tobytes()

child = edge.children.get(key)
```

这段代码实现了一个与普通确定性树搜索不同的机制。

假设在状态 $s$ 下选择动作 $a$。

第一次模拟:

$$s \xrightarrow{a} s_1'$$

第二次模拟:

$$s \xrightarrow{a} s_2'$$

两个随机后继可能不同。

代码以实际后继状态为键，将它们保存为同一个动作下的不同子节点。

如果之前没有见过这个后继:

```
if child is None:
    child = Node()
    edge.children[key] = child
    tail = self._rollout(successor, remaining - 1)
```

新建子节点，并通过 Rollout 估值。

如果之前已经见过:

```
else:
    tail = self._search(
        child, successor, remaining - 1, depth + 1
    )
```

递归进入已有搜索树，继续探索。

你应该从这里建立一个直觉:

**MCTS 不是假设无人机一定执行成功，而是在一棵不断增长的树中积累不同随机结果的信息。**

这也是为什么它能处理运动不确定性。

## 八、第四阶段：Backpropagation —模拟结果怎样更新 Q?

现在一条模拟轨迹已经结束，我们得到一个回报估计。

接下来要把这次得到的信息反馈给搜索树。

你的 _search():

```
value = float(reward) / self.scale + self.gamma * tail

edge.add(value)
node.visits += 1

return value
```

最核心的数学公式是:

$$\boxed{G = \frac{r}{\kappa} + \gamma G_{\text{future}}}$$

假设某个动作经过三次模拟，得到归一化回报:

$$0.2, \quad 0.4, \quad 0.1$$

其平均价值为:

$$Q(s,a) = \frac{0.2 + 0.4 + 0.1}{3} \approx 0.233$$

第 4 次模拟得到:

$$G_4 = 0.5$$

那么新的价值:

$$Q_{\text{new}} = \frac{3(0.2333) + 0.5}{4} = 0.3$$

实际上不需要保存之前的所有回报。

可以增量更新:

$$\boxed{Q_{\text{new}} = Q_{\text{old}} + \frac{G - Q_{\text{old}}}{n_{\text{new}}}}$$

你的 Edge.add() 实现的正是这个公式。

另外，它还使用 **Welford's Recurrence** 更新回报的离差平方和，供最后的根节点动作切换使用。

这里有一个重要概念区别:

**这不是 Q-learning**。

Q-learning 的典型更新需要通过估计的下一状态最优动作价值来构造 TD Target；这里的 Q 值主要来自树搜索和 Rollout 产生的 Monte Carlo 回报样本。

## 九、整套 MCTS 在 _search() 中怎样串起来?

现在你已经理解四个阶段，可以直接阅读简化后的整体逻辑。

```
def search(node, state, remaining, depth):
    if terminal_or_horizon_reached(state, remaining):
        return 0.0

    if depth >= TREE_DEPTH:
        return rollout(state, remaining)

    # 1. Expansion: admit new actions when allowed
    maybe_add_guided_action(node, state)

    # 2. Selection: select an admitted action
    action = select_by_unvisited_or_UCT(node)

    # 3. Sample one stochastic successor
    next_state, reward, done = simulate(state, action)

    if done:
        future = 0.0
    elif successor_is_new(node, action, next_state):
        create_successor_node(...)
        future = rollout(next_state, remaining - 1)
    else:
        future = search(
            child_node(...),
            next_state,
            remaining - 1,
            depth + 1
        )

    # 4. Backpropagation
    value = reward / SCALE + GAMMA * future
    update_action_mean_and_variance(node, action, value)
    node.visits += 1

    return value
```

以上是教学用伪代码，省略了真实代码中的时间截止检查、动作唯一性检查等工程细节，并非可以直接替换的函数。

你可以把一次搜索理解为:

选择一个动作 → 采样一个随机未来 → 估计后续价值 → 更新当前动作评价

这就是整个 MCTS 的核心循环。

## 十、最容易忽视的地方：搜索结束后，为什么不直接执行 Q 最大的动作?

这是你当前实现比较特别的部分，也是我认为需要真正理解的地方。

通常介绍 MCTS 时，最后可以选择访问次数最多或估计价值最高的根节点动作。

但你的代码采用了:

**Anchored Root Decision（以贪心策略为锚点的根节点决策）**。

原理如下。

假设:

- $a_g$：风险感知 Greedy 推荐的动作。
- $a$：MCTS 搜索发现的另一个动作。

你不希望仅仅因为 $Q(s,a)$ 比 $Q(s,a_g)$ 高一点，就轻易放弃已有的强启发式策略。

因为 Monte Carlo 估计存在随机波动。

因此代码定义：

$$m(a) = Q(a) - Q(a_g) - z\hat{\sigma}\sqrt{\frac{1}{N(a)} + \frac{1}{N(a_g)}}$$

其中：

$$z = 1$$

只有当：

$$m(a) > 0$$

时，才考虑切换到其他动作，并在正裕量中选择最大的那个。

如果没有这样的动作，就继续执行 Greedy。

它的设计思想是：

**Greedy 提供可靠基线；MCTS 负责发现足以证明自己更有价值的替代动作。**

但是这里的方差阈值只是一种启发式保护机制。MCTS 样本受自适应选择影响，所以不能把它直接视为严格的统计显著性检验或最优性证明。

实际代码还有两个后备条件：如果 Greedy 没被搜索访问过，按访问次数优先选择已访问动作；如果没有完成任何搜索试验，则直接返回 Greedy。

这是一个很好的研究启发：

**采样预算有限时，合理的决策规则有时与搜索本身同样重要。**

在你 B4 的消融中，切换回 Most-visited 根节点规则会使惩罚完成步数平均增加约 3.2 步，这为当前 Anchored Root Decision 提供了实证支持。

## 十一、最后：Online Planner 怎么控制时间?

对应 plan()：

```
fallback = self._proposal(state, randomize=False, remaining=remaining)
root = Node()

while (
    completed < self.max_simulations
    and remaining > 0
    and time.perf_counter() < self.deadline
):
    self._search(root, state, remaining, 0)
    completed += 1
```

当前设置：

$$K_{\max} = 2000, \qquad \text{默认规划预算} = 9.5\text{秒}$$

停止条件是：模拟试验次数达到上限，或者接近时间截止点。

这里要特别注意：

**2,000 次 MCTS Trial 不等于调用 simulate() 2,000 次。**

一次 MCTS Trial 里面可能包含多次树内状态转移，以及多步 Rollout。

因此：

$$\text{Generative Model Calls} \geq \text{Completed MCTS Trials}$$

通常会明显更多。

最后，真实环境执行一个联合动作，观察下一状态，然后**重新构建根搜索树**进行下一步规划。

这就是 Replanning，也体现了在线规划的闭环特征。

## 第二讲总结

我们现在可以从高层次重新理解你的算法：

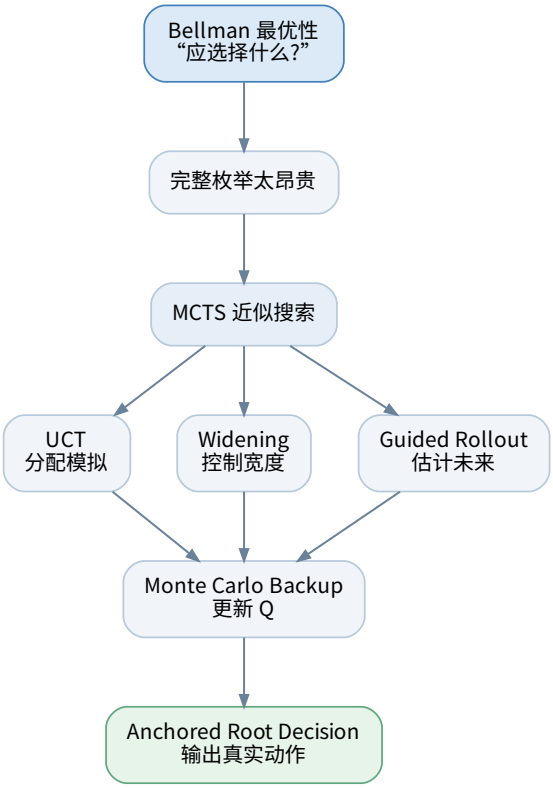



图 7: Bellman 到 MCTS 的层次关系

从课程知识体系看：

| 层次 | 课程概念 | 你代码中的具体实现 |
|---|---|---|
| 数学目标 | Bellman Optimality | 最大化期望折扣回报 |
| 近似方法 | Monte Carlo Tree Search | _search() |
| 搜索选择 | UCT | _search() 中的 UCB |

| 层次 | 课程概念 | 你代码中的具体实现 |
|---|---|---|
| 分支控制 | MCTS Expansion | Progressive Widening |
| 未展开的未来 | Heuristic Rollout | _rollout()、_leaf() |
| 价值估计 | Monte Carlo Backup | Edge.add() |
| 最终决策 | 当前状态下的动作选择 | _anchored()、plan() |

如果只允许你用一句话解释当前算法，我建议这样理解：

**我们使用单机有限时域 Value Iteration 的动作价值提出有希望的联合动作，再利用 MCTS 对它们的随机未来进行有限次数的模拟，通过 Monte Carlo 回报估计长期价值，最后谨慎地选择一个动作真实执行。**

这里还有一道检验题：

假设 MCTS 中某个动作已经被模拟 500 次，平均 $Q = 0.25$；另一个动作只模拟了 5 次，平均 $Q = 0.20$。

**为什么搜索阶段仍然可能优先选择第二个动作，但真实执行阶段未必选择它?**

这个问题能够同时检查你是否理解 UCT 的探索机制，以及 Anchored Root Decision 与 Selection 的区别。

**第三讲我们将专门推导单机 Value Iteration，并从 csr_matrix 到 _value_options()、_proposal()、_leaf() 逐行解释：为什么单机精确 Bellman 计算能让联合 MCTS 更有方向。**

# 第三讲：真正读懂新版 B2——单机有限时域 Value Iteration 如何引导联合 MCTS

本讲针对最新提交 5c7e9c20 重写。直接对应 planner.py 第 72–238、277–407 行，报告 Q2 第 75–112 行和 Appendix「Algorithm definitions」「Value-iteration versus Dijkstra guidance」。特别注意：**新提交的默认算法是 VI Guidance，而不是旧版 Dijkstra Guidance**。

前两讲已经回答了两个问题：什么是 MDP，以及为什么联合动作空间太大，需要有限预算的 MCTS。现在进入整份算法最有新意、也最值得你向考官解释的一步。

**我们没有直接对四架无人机做完整 Value Iteration。我们只对每一架无人机单独做精确的有限时域 Bellman Backup，再把所得的价值信息交给联合 MCTS。**

这是一个经典的「以较容易的子问题指导困难的联合问题」的设计。你应当把它理解成 **exact single-drone relaxation + approximate joint online search**，而不是笼统的「VI 和 MCTS 混合」。

## 一、先用一张结构图建立宏观认识

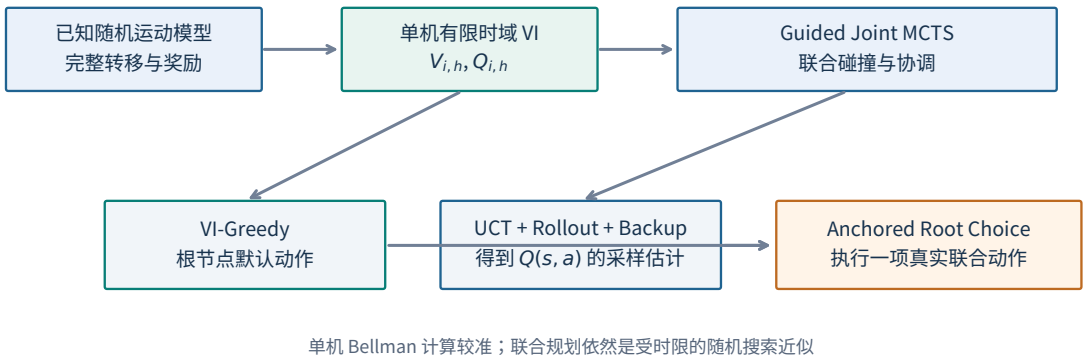



图 8: 新版 VI-guided MCTS 的层级结构

| 计算层次 | 解决的问题 | 代码入口 | 是不是精确的? |
|---|---|---|---|
| 已知随机运动规律 | 一个动作会到哪里，概率是多少? | _slip_model() | 与提供的单机运动分布一致 |
| 单无人机有限时域规划 | 还剩 $h$ 步时，某个动作长期价值多高? | _value_iteration() | 对定义的**单机放松模型**做有限时域 Bellman Backup |
| 联合动作提案 | 哪些四机联合动作值得纳入搜索? | _value_options()、_proposal() | 启发式近似 |
| 联合随机树搜索 | 这些动作在真实联合碰撞语义下表现如何? | _search()、_rollout() | 有限采样近似 |
| 根节点执行决策 | 是否相信搜索结果，放弃 Greedy? | _anchored() | 基于方差的启发式选择 |

**精确与近似的边界一定要划清：**单机 VI 不考虑其他无人机，所以它的最优 $V_i$ 并不是联合最优 $V^*(s)$；MCTS 才负责对联合行动的相互影响进行采样估计。

## 二、为什么 BFS / Dijkstra 不够?

我们先回顾上一版，而不是把它当成「错误的算法」。

**BFS：**如果格子之间的边代价都是 1，计算到目标的最少步数 $\delta_i(p)$。它知道障碍物在哪里，却不知道随机偏移概率。

**Risk-aware Dijkstra（旧版 A）：**将下一格的近似成本设为

$$\boxed{c(q) = 1 + \lambda\rho(q)}$$

其中 $\rho(q)$ 是从 $q$ 出发时随机落到障碍物的局部概率，$\lambda$ 将危险性转换成「等价步数」；Dijkstra 计算确定性加权图的最小路径成本 $J_i(p)$。

它比 BFS 更有风险意识，但存在三个理论局限：

1. 一条可执行动作真正有很多随机后继，而 Dijkstra 每条边只对应确定性图边。
2. 真正的优化目标包含**折扣目标奖励、碰撞惩罚和有限时域**，并不是固定的「距离 + 风险」之和。
3. 一段绕路虽然短期安全，却可能消耗剩余时域；Dijkstra 的静态成本本身不随「还剩几步」改变。

因此新版转向：**既然老师已经给出了随机转移规律，而单机状态空间又比较小，何不直接在单机 MDP 上使用 Bellman Backup?**

这是算法改进真正的理论动机。

## 三、先把老师的随机移动规律写准确

让当前位置为 $p$，每架无人机有 $B = 8$（固定高度）或 $B = 26$（三维）个方向。记 $d(p)$ 为仿真器给出的距离场，噪声强度是 $\alpha$。

原始的「按命令运动」权重为：

$$\boxed{\pi(p) = \frac{d(p)}{d(p) + \alpha}}$$

如果想执行的方向是 $a$，那么该方向先分配权重 $\pi(p)$，其余 $B - 1$ 个物理方向各分得

$$\frac{1 - \pi(p)}{B - 1}.$$

注意：**越界方向被删去并重新归一化，障碍物格子却是合法的随机落点结果，落上后判定碰撞**。障碍物不应和越界方向一起提前删除，否则会低估碰撞风险。

设 $V(p)$ 为当前格子的有效**物理邻居方向数**（在网格内、可包含障碍物），则归一化总质量是：

$$M(p) = \pi(p) + (V(p) - 1)\frac{1 - \pi(p)}{B - 1}$$

这里假设**所请求的动作本身指向网格内格子**。于是：

$$\boxed{\ell(p) = \frac{\pi(p)}{M(p)}, \qquad \sigma(p) = \frac{1 - \pi(p)}{(B - 1)M(p)}}.$$

| 符号 | 物理含义 | 代码 |
|---|---|---|
| $\pi(p)$ | 边界归一化之前的意图成功权重 | _slip_model() 的 success |
| $M(p)$ | 保留网格内方向后的概率总质量 | mass |
| $\ell(p)$ | 最终的意图方向概率 | self.land_intended |
| $\sigma(p)$ | 每一个其他合法物理方向的偏移概率 | self.land_slip |
| $\rho(p)$ | 下一步随机落入障碍物的概率，**只对合法目标命令** | self.departure_risk |

直接看源码核心（做了不影响逻辑的排版）：

```
alpha = float(self.cfg.alpha)
success = self.clearance / np.maximum(self.clearance + alpha, 1e-12)
slip = (1 - success) / max(1, len(self.physical_vectors) - 1)
valid = self._neighbour_sum(np.ones(self.shape, dtype=bool))
mass = np.maximum(success + (valid - 1) * slip, 1e-12)
self.land_intended, self.land_slip = success / mass, slip / mass
self.departure_risk = self._neighbour_sum(~free) * self.land_slip
```

这段逻辑不仅用于可视化风险地图，还给 VI 的模型提供了独立核对的数学依据。

#### 一个小算例：边界归一化为什么重要?

假设每架无人机可选 8 个平面方向，某格子的意图权重为 0.7，但只有 5 个方向落在网格内；命令本身合法。

$$M = 0.7 + 4 \cdot \frac{0.3}{7} \approx 0.87143.$$

因此意图动作的真实概率约为 $0.8033$，而**不是** $0.7$。被删除的越界方向质量会按规则重新分配。

这是 B1、B2 与代码一致性检查中很容易失分的一处。你的单元测试 test_slip_model_matches_simulator_probabilities 专门覆盖了它。

## 四、有限时域 VI：时间不是可有可无的下标

我们现在正式建立单无人机子问题。

设有 $n$ 个自由格子，目标为 $g_i$，还剩 $h$ 步；成功到达目标或碰撞都会终止该**单无人机**的规划过程。

回报与老师的奖励一致：每执行一步承担 $c_{\text{step}}$，撞障碍罚 $C$，首次到达目标奖 $G$。给定单机随机转移 $P_i(p' \mid p, a)$，令 $P_{\text{crash}}(p, a)$ 表示碰撞概率。

$$\boxed{Q_{i,h}(p,a) = c_{\text{step}} + C\,P_{\text{crash}}(p,a) + G\,P_i(g_i \mid p,a) + \gamma \sum_{p' \in \text{Free}} P_i(p' \mid p,a) V_{i,h-1}(p')}$$

同时：

$$V_{i,h}(p) = \max_a Q_{i,h}(p,a), \quad V_{i,0}(p) = 0, \quad V_{i,h}(g_i) = 0.$$

这里需要特别理解三个技术细节。

**第一：为什么 $V_{i,h}(g_i) = 0$?** 因为到达目标的奖励已经在“进入目标的那一步”支付；一旦单机到达，单机子问题结束，不能在后续每一步重复获得目标奖励。

**第二：碰撞分支为什么没有未来价值?** 因为它是终止事件。代码中稀疏转移矩阵只保留落在自由格子的转移概率；落障碍的质量单独用于碰撞惩罚。

**第三：$h$ 是剩余时域，而不是普通状态标签。** $Q_{i,1}$、$Q_{i,5}$、$Q_{i,60}$ 可以给出不同动作排序；这是旧版静态 Dijkstra 无法直接表达的。

#### 用一个数值例子让公式真正“活”起来

设 $\gamma = 0.9$、$c_{\text{step}} = -1$、$C = -100$、$G = 100$。某动作有 80% 概率直接到目标、15% 概率到普通安全格子、5% 概率撞障碍，普通格子后续价值 $V_{h-1} = 20$。

则该动作的价值为：

$$Q_h = -1 + 100(0.8) - 100(0.05) + 0.9(0.15)(20) = 76.7.$$

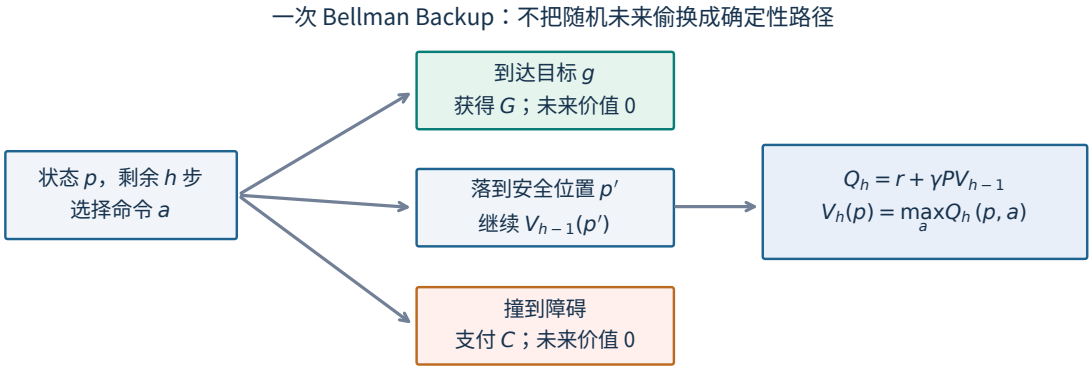



图 9: VI 的一层 Bellman Backup 示意

不要把这当作真实课程环境的测量值。它是**教学算例**，目的是解释随机概率如何同时决定奖励和未来价值。在真正代码里，Bellman Backup 反复从 $h = 1$ 计算到 $H$，所有自由格子和动作同时更新。

## 五、从 Bellman 公式到 scipy.sparse：源码逐段拆解

这是你需要准备口试解释的一段核心实现。_value_iteration() 在 planner.py 第 152–195 行。

### 第一步：给自由格子建立连续索引

```
cells = np.argwhere(self.free)
index = np.full(self.shape, -1, dtype=np.int64)
index[tuple(cells.T)] = np.arange(len(cells))
n, B, A = len(cells), len(self.physical_vectors), self.base
```

原本三维坐标是 $(x, y, z)$，为了矩阵运算要转成 $0, 1, \ldots, n-1$ 的行索引。障碍格子用 $-1$ 表示不进入正常状态集合。

请注意，B 是**完整物理方向集合**，A 是该次实验允许给出的**命令动作集合**。在 6/18/26 方向的控制实验中，两者不一定相等。

### 第二步：对每个状态—动作枚举所有物理落点

```
weights = np.repeat(
    ((1 - success) / max(1, B - 1))[:, None], B, axis=1
)
weights[:, j] = success
weights[~valid] = 0.
weights /= np.maximum(weights.sum(axis=1, keepdims=True), 1e-12)
crash[a::A] = (weights * (valid & ~lands)).sum(axis=1)
```

你应当能说清楚：

- weights[:, j] = success：命令对应的物理方向获得意图权重；
- weights[~valid] = 0：删除越界方向；
- weights /= ...：按老师的规则重新归一化；
- valid & ~lands：落在网格内、但**并非自由格子**，所以是撞障碍的概率质量。

这一步是把 B1 的 $T$ 从数学定义落实成可计算的稀疏矩阵。

### 第三步：只存非零转移，避免密集 $n \times n$ 矩阵

```
P = csr_matrix(
    (np.concatenate(probs),
     (np.concatenate(rows), np.concatenate(cols))),
```

```
    shape=(n * A, n)
)
```

形状是 $(nA, n)$，因为每个状态–动作对占一行，目标状态占一列。

**为什么用 CSR Sparse Matrix?** 一个物理动作只可能到附近有限个格子，并不是从每个格子都随机跳到其他任意格子。稀疏表示不会保存大量零元素，可以把 Bellman 更新写成快速的矩阵向量乘法。

### 第四步：真正的 Bellman Backup

```
reward = step + hit * crash + (
    prize * P[:, g].toarray().ravel() if g >= 0 else 0.
)
V, history = np.zeros(n), [np.zeros(n)]
actions = [np.zeros((n, A), dtype=np.float32)]
for _ in range(horizon):
    Q = (reward + self.gamma * (P @ V)).reshape(n, A)
    V = Q.max(axis=1)
    if g >= 0:
        V[g] = 0.
    history.append(V)
    actions.append(Q.astype(np.float32))
```

每一轮恰好执行：

$$Q_h \leftarrow r + \gamma P V_{h-1}, \qquad V_h \leftarrow \max_a Q_h.$$

其中 history[h] 就是 $V_{i,h}$，actions[h] 就是 $Q_{i,h}$。保存整个历史而不是只保存最后的 $V_H$，是为了让在线规划在每一步使用**剩余时间对应**的动作价值。

### 第五步：什么时候回退到旧版 Dijkstra?

```
if guidance == 'vi' and self.free is not None and self.free.size <= 20000:
    self._value_iteration()

if n * A * (horizon + 1) * len(self.goals) > 2.5e7:
    return
```

当网格过大或保存全部 $Q_h$ 太昂贵时，self.q_history 保持空值；后续 _options() 就回退到之前的静态 self.costs，也就是 Dijkstra 引导（再大的网格则退回 Chebyshev 近似）。

**这意味着默认标志是 'vi'，不等于任何尺寸的地图都实际运行 VI**。口试中说成“所有地图均由 VI 精确求解”是不准确的。

## 六、_value_options()：怎样把最大化 Q 变成最小化动作成本?

单机 VI 计算出的 $Q_{i,h}(p, a)$ 越大越好，而你前一版 _proposal() 是以“越小越好”的分数排序。

所以新版使用：

$$\boxed{\text{score}_i(a) = -\frac{Q_{i,h}(p_i, a)}{\eta} + 0.025\,\|p_i + \Delta(a) - g_i\|_1 + \text{interDronePenalty}_i(a)}$$

其中 $\eta = \max\{|c_{\text{step}}|, 10^{-3}(G + |C|), 10^{-9}\}$；中间的曼哈顿距离是**微弱的同分偏好**，不是另一个 VI 奖励项。

查看代码：

```
h = longest if remaining is None else max(1, min(longest, int(remaining)))
```

```
q = self.q_history[i][h][self.value_index[cell]]
scores = np.where(valid, -q / self.risk_unit + tie, INVALID)
```

请记住三点:

1. $-Q$ 是为了用升序选择最优行动;
2. $h$ 是真实剩余时间，不是总时域常量;
3. 仍然过滤越界或意图目标为障碍的动作，但不把随机撞墙概率从 VI 模型中删掉。

这就是为什么**静态的风险地图只是诊断图，真正用于 VI 的是整个转移分布**。

## 七、联合动作生成：VI 本身不懂其他无人机，怎么办?

这是本算法最重要的近似层。

对每一架活动无人机，我们已经知道单机 $Q_{i,h}$。但联合动作是 $(a_1, \dots, a_N)$，其他无人机的目标格子可能被占据、还可能发生随机偏移。

你的 _proposal() 采取**顺序构造联合动作**的方式:

```
cells = [tuple(map(int, p)) for p in state[:, :3]]
reserved = {cells[i] for i in np.flatnonzero(reached)}
active = np.flatnonzero(~reached)
if randomize or explore:
    active = self.rng.permutation(active)

for i in active:
    targets, cost, _ = self._options(int(i), state[i, :3], remaining)
    cost = cost.copy()
    for j, target in enumerate(targets):
        if target in reserved:
            cost[j] += RESERVED
    # 之后按 Greedy / Sample / Explore 规则选择
    ...
    reserved.add(targets[chosen])
```

为什么要 reserved? 因为如果两架无人机都计划落到同一格子，联合动作很可能触发老师的同格碰撞规则。因此已经选过的目标格子会被临时预约。

**但这不意味着保证无碰撞:** 随机偏移仍可能把两架无人机带到同一个落点; 顺序预约也不证明不存在交换以外的复杂冲突。

### 机间风险的额外惩罚

代码 _slip_penalty() 对仍在运动的其他无人机添加一个局部、近似的风险附加项:

$$\text{penalty}_i(a) \approx \lambda \ell(p_i) \sum_{k \neq i,\ r_k = 0} \sigma(p_k)\, \mathbf{1}\{p_i + \Delta(a) \in \mathcal{N}(p_k)\}.$$

它是一种**动作提案代理目标**，而不是完整的联合碰撞概率。它利用已知的偏移分布偏好互相不干扰的候选动作。

### Greedy、Sample、Explore 的分工

| 机制 | 何时使用 | 为什么需要 |
|---|---|---|
| Greedy | 根节点第一个候选 / 最终锚点 | 保留可靠的 VI 基础策略 |
| Sample | 其余候选中的主要采样形式 | 按分数在前四个、分数相差不超过 1.05 的动作中进行 softmax |

| 机制 | 何时使用 | 为什么需要 |
|---|---|---|
| Explore | 以 0.15 概率用于候选扩展 | 防止只围绕单机偏好，缺少联合协调方案 |
| Rollout Explore | 概率 0.10 | 在模拟未来时保留少量探索，而非每步完全贪心 |

对应到你的 MCTS：**VI 决定「先试哪些动作」，UCT 决定「给已纳入的动作分配多少试验」**。它们并不互相替代。

## 八、_leaf()：这次也升级成 VI Value，而不只是动作提案

旧版 _leaf() 使用 BFS 最短步数估算未观察到的未来收益；这会对墙壁缺口等随机危险场景过于乐观。

最新版保留 BFS 用于几何连通性和部分回退逻辑，但如果 VI 成功预计算，就主要使用：

$$\widehat{V}_{\text{joint}}(s,h) \approx \sum_{i:\, r_i=0} V_{i,h}(p_i) + \widehat{C}_{\text{reached}}(s,h)$$

另外，如果已到达目标的无人机冻结在狭窄通道，造成其他无人机的静态路线完全被截断，代码 _cut_off() 会识别这种情况，并用不可能继续得到目标奖励的步成本代替。

但这里有一个微妙的问题：**每架无人机到达目标后不再移动，却仍在联合任务执行期间贡献每步成本**。所以不能仅仅把各个单机 $V_i$ 求和就假装得到联合真实价值；代码还对已完成任务的无人机补了近似的后续时间成本。

实际源码：

```
if self.values is not None:
    h = min(int(remaining), len(self.values[0]) - 1)
    stuck = h if self.gamma == 1 else (1 - self.gamma**h)/(1-self.gamma)
    value = self.cfg.step_cost * (self.count - len(distances)) * costs
    for i in range(self.count):
        if not state[i, 3]:
            value += (
                self.cfg.step_cost * stuck if self._cut_off(i, state)
                else self.values[i][h][self.value_index[tuple(state[i, :3])]]
            )
    return float(value) / self.scale
```

这是近似估值，而非联合 MDP 的精确 Bellman 解。尤其是**单机最优路径不可能自动考虑尚未冻结的其他无人机未来占路**。

## 九、为什么这次 VI 升级比「换一种启发式」更深?

你应当能清晰回答：

1. **Dijkstra 的输入**是加权确定性图；**VI 的输入**是随机转移概率和真实奖励。
2. **Dijkstra 的输出**主要是到目标的静态风险加权距离；**VI 的输出**是随剩余时域变化的 $Q_{i,h}$ 与 $V_{i,h}$。
3. **Dijkstra 只引导动作**；新版 VI 同时引导动作扩展、Rollout 和叶节点估值。
4. **Dijkstra 仍有价值**：在预计算预算不可承受时作为回退；在 B4 中作为明确的算法基线。
5. **VI 本身没有解决联合耦合**；MCTS 的真实生成模型仍用于多无人机随机相互作用。

这意味着两层 Bellman 思想正在合作：**较小子问题离线精确 Backups, 联合问题在线随机采样 Backups**。这恰好体现了课程中「已知模型可以用于规划」的核心思想。

## 十、课堂演练：你能推导这三个问题吗?

**问题 A：**如果 $\alpha \to 0$，VI 和 Dijkstra 在某些简单无障碍任务中为什么可能趋于同一路径？提示：随机后继概率集中到意图方向，但还要考虑折扣与奖励。

**问题 B：**如果 $N = 4$、每架单机 VI 已经精确，为什么不能宣布联合策略也精确？提示：目标占用、同格碰撞、共享任务终止时间。

**问题 C：**一个新动作比 VI-Greedy 的单机 $Q$ 低，但 MCTS 为什么仍可能发现它是更好的**联合动作**？提示：主动让路、冻结目标、为另一个无人机避免冲突。

### 第三讲结论

**新版的核心不是「更复杂的 Dijkstra」，而是利用已知随机转移模型，提前求出每架无人机的有限时域 Bellman Value，再把它作为 MCTS 的低成本、强信息量先验。**

真正理解这一点，才能在口试中解释新版 VI 的贡献，也才能看懂 B4 为什么既肯定 VI 的平均效率改善，又承认 MCTS 的额外收益可能不稳定。

# 第四讲：B3——从 plan() 到 step() 的 Python 闭环系统（8 分）

本讲是新增内容。对应提交入口 run_u8076655.py、environment.py、multi_drone.py、planner.py。B3 的正式得分来自现场程序表现。

我们已经建立数学模型并了解规划算法。下一步要回答：**一个数学算法究竟怎样变成在老师机器上能够运行、满足 10 秒限制的真实程序?**

## 一、首先区分三种程序角色

| 代码对象 | 真正职责 | 不应该混淆的事情 |
|---|---|---|
| execution | 保存本次任务的真实随机执行状态 | 不应该被树搜索反复推进 |
| model | 为规划器提供独立的 simulate() 随机后继 | 不是另一个需要完成任务的真实无人机系统 |
| planner | 在观测状态下输出一个联合动作整数 | 它并不直接修改执行环境 |

打开 run_u8076655.py，可看到:

```
execution = make_env(config, seed=seed, visualize=visualize)
state = execution.reset()

model = make_env(config, seed=1000003 + seed)
model.reset()

planner = MCTSPlanner(model, seed=2000003 + seed)  # 默认 guidance='vi'
```

真实执行、规划时模拟、动作提案采用不同的随机数流。这个细节很专业：如果搜索消耗的随机数会影响真实未来，算法在做更多假想模拟后可能无意中改变测试任务的随机结果，削弱实验可比性。

因此:

$$\boxed{\text{真实执行 RNG} \neq \text{规划模拟 RNG} \neq \text{动作提案 RNG}}$$

这里的不同指随机数流被隔离，而不是随机运动规律不同。执行与模拟使用相同的转移机制、不同的采样序列。

## 二、初始化时的 VI 与每步在线 MCTS 是两项不同的计算

最新版 MCTSPlanner(...) 的默认参数已经是 guidance='vi'。构造规划器时先预计算自由格子、运动概率与单机有限时域价值表，之后每个真实 plan() 才使用这些 $Q_{i,h}$ 与 $V_{i,h}$ 开始联合随机搜索。

这一点与旧版讲义不同：**_value_iteration() 是初始化阶段的预计算，_search() 才是每个真实决策重建的在线搜索**。

当前四组 Demo 的初始化记录约在 0.01–0.18 秒范围，均很短；但它们是所记录机器上的数值，不是任意大地图的预计算成本上界。当格子或时域过大，代码会回退到静态 Dijkstra 或进一步的几何近似。

## 三、一次 Episode 在代码中怎么走?

```
for step in range(cfg.max_num_steps):
    if success or failure:
        break

    action = planner.plan(state, planning_time)
    successor, reward, done, info = execution.step(action)
```

```
    total_reward += float(reward)
    discounted_return += cfg.discount_factor**step * float(reward)
    state = successor
```

把它翻译成自然语言：

1. 从真实环境获得当前联合状态；
2. 在有限预算内运行 planner.plan()；
3. 规划器返回一个整数编码的联合动作；
4. execution.step(action) 真正执行一次随机转移；
5. 记录真实奖励、新状态和是否成功；
6. 如果任务尚未结束，重复从第 1 步开始。

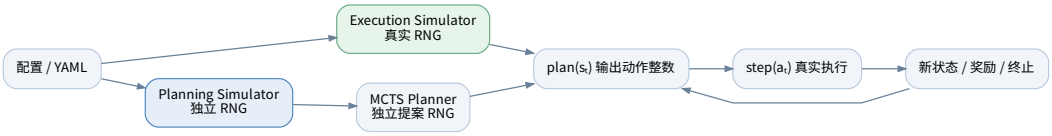



图 10: B3 双仿真器闭环

这就是在线规划系统最基本的闭环。simulate() 只是**假想未来**，step() 才是**真实任务推进**。

## 四、为什么代码中的 simulate() 还要知道剩余时间?

你的原始环境中，step() 会维护内部时间步并检查 max_num_steps；但是 simulate(state, action) 本身只模拟单次转移，并不负责推进真实时间计数。

因此，如果规划器在无限深度搜索中不处理剩余时域，就可能在假想未来里多模拟了本来已经没有的执行步数。

你在 plan() 中：

```
remaining = max(
    0,
    int(self.cfg.max_num_steps) - self.steps
)
```

以及 _search(..., remaining, depth) 中逐层递减 remaining，保证规划逻辑能够反映有限时域。

这也是第一讲“数学状态中要包含 $t$”在代码层面的落地。

## 五、10 秒规则不是一句口号，而是一个接口契约

老师 B3 的明确约束是：**每次实际决策最多 10 秒规划时间，不允许并行化**。

你的程序默认给每次规划：

$$\text{planning budget} = 9.5 \text{ s}.$$

在 plan() 中：

```
start = time.perf_counter()
budget = min(budget, self.budget_cap)
self.deadline = start + max(0., 0.97 * budget - 0.002)
```

也就是说，搜索不会主动用满全部 9.5 秒，而是预留一些时间作为安全余量。_simulate() 入口也会检查 deadline:

```
if time.perf_counter() >= self.deadline:
    raise Deadline
return self.env.simulate(state, action)
```

搜索到时间后，会中止当前试验并使用已完成的统计结果；如果没有完成试验，则返回已经准备好的 Greedy Fallback。

**工程上要区分“主动检查截止时间”与“绝对硬实时保证”**。如果某次模拟调用自身耗时很长、机器负载大，停止检查只能在函数调用前生效，不能在调用的中途任意打断。因此正式 Demo 仍应在老师的实验室机器上验证。

#### 为什么还有 max_simulations = 2000?

一旦简单环境能够非常快地完成 2,000 次 Trial，就无需为了等满 9.5 秒而继续无意义地重复搜索。于是两个限制并存:

停止 = 达到搜索次数上限，或到达时间截止

而且一条 Trial 可能调用 simulate() 很多次，因为包含树内转移和 Rollout，因此“2000 Trial”不等于“2000 次随机运动采样”。

## 六、为何每个真实时间步都重建搜索根节点?

plan() 每次运行都会:

```
fallback = self._proposal(state, randomize=False, remaining=remaining)
root = Node()
```

这是典型的 **Receding Horizon / Replanning** 思想：观察新的真实状态后，在这个状态重新考虑未来。

优点是无需精确维护旧搜索树与新状态的一致性；缺点是没有直接复用上一步的 MCTS 统计数据。

在随机环境里，原来计划的轨迹经常不会原封不动地出现。在线重新规划可以应对这一事实。

## 七、从日志读取真实算法行为

报告 B3 使用 Open 2D 例子给出的部分日志:

```
step   1/80 | 0/1 at goal | reward    -1.0 | plan  7.42s  2000 sims | (1,3,0)
step   2/80 | 0/1 at goal | reward    -1.0 | plan  7.26s  2000 sims | (2,3,0)
...
step  14/80 | 1/1 at goal | reward    99.0 | plan  1.84s  2000 sims | (11,5,0)*
success: True (success), steps: 14, total discounted reward: 64.58
```

读懂每一列:

- step：真实环境已经执行的步数;
- 0/1、1/1：已经到达目标的无人机数 / 总数;
- reward：本次真实转移产生的采样奖励;
- plan：本次规划耗时，而非全部 Episode 耗时;
- sims：完成的搜索 Trial 次数;
- 末尾坐标：本次真实移动后的无人机位置。

* 标记该无人机的 reached 状态。

这段日志尤其说明：**2000 次试验都用于决定当前的一个真实动作**。而下一时间步又进行新一轮搜索。

## 八、--visualize 与科研图表分别能看到什么?

当前代码有两类可视化。最新版的风险图和完成轨迹图已根据 VI 更新，读图时要区分「理论价值热力图」与「一次真实执行轨迹」。

**真实执行场景（Vedo 交互图形）**：显示障碍物、目标、无人机和实际执行轨迹。它回答“无人机最后怎么走”。

**离线分析图**：场景图集、风险热图、组件消融和预算曲线。它们回答“策略为什么这样设计，以及平均效果如何”。

你可以使用：

```
python run_u8076655.py \
  --config environments/demo_four_3d.yaml \
  --visualize
```

渲染窗口是解释工具，不应该被包含在 10 秒 MCTS 模拟的计算主循环里。

### 一条十分重要的实验伦理原则

**不能把 MCTS 内部的假想轨迹当作真实 Episode 的结果**。真正用于计算成功率、奖励和完成步数的，是 execution.step() 所产生的真实轨迹。

## 九、B3 的正确性还依赖哪些边界条件?

真实代码在进入规划循环前，检查目标是否重复、静态路径是否不可达。它还需要正确处理：

- 边界上的动作归一化：不能假设每个方向总是等概率合法；
- 到达目标后冻结：已完成任务的无人机仍然占据该格子；
- 静态障碍碰撞：真实模型可能让无人机留在原位置，但碰撞已导致任务结束；
- 无人机间碰撞：使用实际下一格占用而不是几何线段相交；
- 已达到步数上限：仿真 Episode 必须终止；
- 缺失可视化依赖：无图形环境下也应能以命令行运行。

这些属于 B3 的工程正确性；即使 MCTS 理论完全正确，忽略其中一项也可能导致现场失分。

## 最新版发布验证：哪些数据应该记住?

| 默认 Demo | 是否成功 | 步数 | 最大单步规划耗时 |
|---|---|---|---|
| Open 2D（单机） | 成功 | 14 | 7.42 秒 |
| 老师双机示例 | 成功 | 17 | 7.48 秒 |
| Strong wind（单机） | 成功 | 30 | 6.34 秒 |
| Four 3D（四机、五障碍） | 成功 | 13 | 9.21 秒 |

四组 Demo 共 74 次真实决策，其中 73 次达到 2,000 Trial；另一次因截止检查完成 1,815 次。results/review_validation.json 记录 **45 项单元测试**通过，另有一次 10 秒限制的强制检查，实际约 9.70 秒。

这里的 Demo 步数是**固定 seed 的一条执行轨迹**，不能用 14 vs 11、30 vs 18 直接评价 VI 是否优于 Dijkstra；多种子配对结果才是正确的版本性能比较。

## 第四讲总结

**B3 的本质，是把数学规划算法封装成一个可测试、可重复、受时间约束的闭环决策系统**。

你要能从代码中清楚指出三条界线：

$$\boxed{\text{初始化的单机 VI 预计算} \neq \text{搜索时的随机模拟} \neq \text{真实执行}}$$

# 第五讲：B4（20 分）——怎样用实验真正检验新版 VI-MCTS?

本讲对应报告 Question 4 的 Experimental design、Appendix「Development-set parameter selection」「Value-iteration versus Dijkstra guidance」,以及 experiments.py、scripts/analyse_results.py。所有数字均针对提交 commit 5c7e9c20，不是此前的 Dijkstra 版本。

如果说 B1 是「定义什么叫好决策」，B2 是「如何寻找好决策」，B3 是「怎样正确运行算法」，那么 B4 的真正任务是：**用有控制的实验回答算法为什么有效，以及在哪些条件下不再有效。**

B4 占 Part B 的 20/35。高分绝不是靠多放几张轨迹图，而是要让研究问题、控制变量、统计估计和结果解释构成同一条证据链。

## 一、先分清实验中的四种“次数”

最新版项目中很容易混淆的四种计数：

| 项目 | 规模 | 用途 | 是否直接属于新版正式 B4 评估? |
|---|---|---|---|
| 正式评估 Episode | **3,460** | 回归、全因子、控制、消融、预算、随机、基线和压力测试 | 是 |
| 开发集调优 Episode | **2,880** | 三轮搜索参数与 Guidance 方法选择 | 否，必须与最终评估分开 |
| 新旧 Guidance 配对条件 | **1,680** | Dijkstra A vs VI B 的去重配对比较 | 是两套正式实验的**配对分析**，不是新增加 1,680 个独立任务 |
| 单次规划的 MCTS Trial | 最多 **2,000** | 一个状态下的内部未来模拟预算 | 不是 Episode 数，也不是模型调用次数 |

正式评估的构成：

| Suite | 次数 | 它主要回答什么问题? |
|---|---|---|
| S1 factorial | 540 | 状态、动作与障碍复杂度的主效应及交互 |
| S2 controls | 160 | 几何障碍与随机噪声、动作标签数与控制能力的区别 |
| S3 components | 1,040 | 删去不同组件后性能变化多少 |
| S4 budget | 400 | 搜索 Trial 从 16 到 2,000 时，回报是否改善 |
| S5 generated layouts | 160 | 更广泛的无人机数量与噪声组合 |
| Regression | 360 | 18 种边界条件、特殊场景与已有示例 |
| Search-free baseline | 520 | 联合 MCTS 与其 Greedy 基线的差异 |
| Supplementary cases | 280 | 单机墙壁、曲折路线、分层运动、协作通道 |
| **总计** | **3,460** | 按原始 CSV 的条目数计 |

请注意：报告的旧版和新版 Guidance 比较包含独立的**归档旧结果**，这些不是新版 3,460 次之内又要加一次的 Episode。统计过程中也保留了开发集与评估集的区别。

## 二、S1 全因子：为什么是 $3^3 \times 20 = 540$?

老师要求研究 State Space、Action Space、Clutter 三个因素。

报告做了一个 $3 \times 3 \times 3$ 全因子设计：

| 因素 | 三个水平 | 数学/物理意义 |
|---|---|---|
| 状态空间 S | $n_y \in \{5, 10, 20\}$ | $12 \times n_y \times 3$ 网格；改变自由空间与理论状态数 |

| 因素 | 三个水平 | 数学/物理意义 |
|---|---|---|
| 动作空间 A | 每架 6、18、26 个方向 | 两机联合动作分别为 36、324、676 |
| Clutter C | 0、2、5 个障碍物 | 改变几何绕行与随机碰撞暴露 |

其余条件固定：两架无人机、$\alpha = 0.25$、$H = 60$、每个条件 20 个执行种子。

$$3 \times 3 \times 3 \times 20 = 540.$$

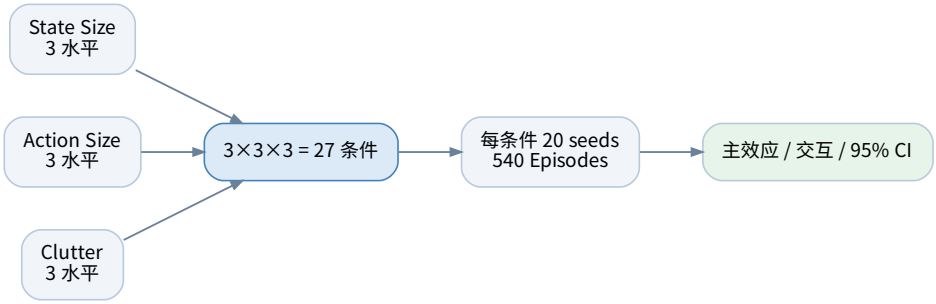



图 11: S1 全因子实验设计

### 主效应不是某一个场景的前后对比

例如 Clutter 主效应：对每个执行种子，先在所有状态和动作设置上平均 5 障碍物与 0 障碍物的表现差，再对 20 个 seed block 做统计。

$$\widehat{\Delta}_C = \frac{1}{20}\sum_{s=1}^{20}\left[\frac{1}{9}\sum_{i,j} L_{s,i,j,C=5} - \frac{1}{9}\sum_{i,j} L_{s,i,j,C=0}\right].$$

它测的是**当前实验水平内**的主效应，不是未来所有地图的普遍定律。

### 交互作用为什么有用?

假设更多动作方向在无障碍时只能快一点，但在复杂障碍地图里快很多，说明 Action 的效果依赖 Clutter。

$$\text{Interaction}(A, C) = \Delta_A(C_{\text{high}}) - \Delta_A(C_{\text{low}}).$$

你报告中除了三个主效应，还讨论了 Action × Clutter、State × Action 等交互，属于 B4 中比较有深度的设计。

## 三、S1 仍然有两类混杂：为什么还需要 S2?

**混杂一：状态变大不等于路径变难**。将 $n_y$ 从 5 增到 20，确实让理论状态数增大，但可能主要增加不在主要路径附近的自由区域。如果平均完成步数几乎不变，不能据此宣称 MCTS 普遍不受状态空间增长影响。

**混杂二：动作数量多不等于搜索分支纯粹更多**。每架从 6 个轴向命令增加到 26 个三维命令，会让机器人得到新的实际运动能力，而不仅是多了几种标签。更丰富的控制可能绕过障碍、缩短距离。

因此 S2 设计了两类对照。

### S2-A：减小噪声，观察“纯几何”问题

将 $\alpha$ 从 0.25 降到 0.01，比较 Clutter 影响。噪声小的时候，障碍增加主要体现为几何绕行；噪声大时还会增加随机偏移碰撞的机会。

### S2-B：只复制动作标签，不改变物理运动规律

将同样的物理动作重复编码成 $\times 1$、$\times 2$、$\times 4$ 倍联合动作标签，让 $|A|$ 从 676 变成 2,704、10,816，但不改变物理转移分布。

如果性能几乎不变，就支持以下**受限结论**：当前的启发式候选过滤与渐进拓宽不会因无意义标签重复而显著恶化。它不是「算法对所有高维动作空间都免疫」的证明。

**计数口径提醒：** S2 新产生 160 条 Episode：低噪声 120 条、复制标签 $\times 2$ 和 $\times 4$ 共 40 条。表格中也引用来自 S1 的 $\alpha = 0.25$ 和 $\times 1$ 参考条件，但这些不重复计入 160 条。

## 四、S3 消融：不是“关掉一个超参数”这么简单

一项组件消融实验就是回答：**如果移除某个实现机制，其他条件尽量保持不变，结果究竟怎样改变?**

你最新版有以下变体：

| Variant | 具体改变 | 不能过度解释的地方 |
|---|---|---|
| no_guidance | 随机动作提案/随机 Rollout，不再执行原有过滤 | 同时移除了引导与意图落点合法性过滤；不能把全部差异归因于单机 VI |
| no_leaf | 截断估值设为 0 | 只检验现有搜索深度与这些场景下的边际增益 |
| no_widening | 不设渐进动作数量上限 | 不是证明宽搜索总是更好或更差 |
| no_risk | 取消机间风险附加项 | 单机 VI 的障碍碰撞概率仍然存在，不能把它误读为关闭**所有**风险意识 |
| visits_choice | 根节点选访问次数最多的动作 | 比较最后如何执行动作，不是修改 UCT 本身 |
| max_q | 根节点选估计均值 $Q$ 最大的动作 | 说明采样噪声下的“最大均值”可能并不稳健 |
| heuristic | 令 MCTS 搜索次数为 0 | 检验强 VI 基础策略与联合搜索的**增量差异** |

S3 在三个因素的高/低端点上选择 8 个组合；no_guidance 每格 10 种子，其余各 20 种子，因此总数为：

$$8 \times (10 + 6 \times 20) = 1,040.$$

**最重要的实验伦理点：** no_guidance 会使某些动作过滤规则也消失，因此它只能证明「整个 Guided Proposal + Rollout 体系」重要，不能孤立地证明 VI 独自贡献了全部效果。

## 五、S4 搜索预算：为什么更大的计算预算不保证更好的表现?

四种任务，每种研究五种 Trial cap：

$$K \in \{16, 64, 256, 1024, 2000\}.$$

每个条件 20 种子：

$$4 \times 5 \times 20 = 400.$$

MCTS 搜索次数多，应该帮助价值估计收敛吗? 在充分条件下，某些 UCT 变体具有渐近性质；但**有限的**

**2,000 次模拟、巨大联合状态空间、启发式叶值和最终锚定规则**都不支持简单的“预算越大完成率必然越高”。

应该同时记录成功率、惩罚完成步数 $L$、根节点纳入动作数量和新节点平均深度，而不只看运行时间。

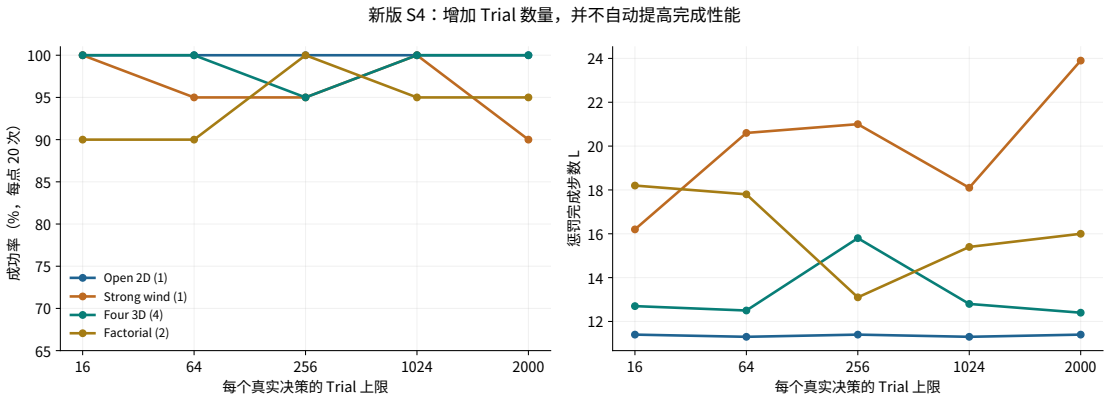



图 12: 搜索预算与完成性能的关系：教学复现图

## 六、S5 随机生成：扩大覆盖，但不能混淆变量

$N \in \{1, 2, 3, 4\}$，$\alpha \in \{0.05, 0.25, 1, 2\}$，共 16 种组合，每组一**个布局**与 10 个随机执行种子：

$$4 \times 4 \times 10 = 160.$$

这可以探索更难的四无人机、高噪声情形；但无人机数量上升时，状态空间、动作空间和协调要求一起增长，无法凭这 160 条样本把三种效应独立分离。

## 七、统计部分：成功率、完成步数、配对差与 CI 不一样

对于每次 Episode，定义：

$$L = \begin{cases} \text{真实完成步数}, & \text{成功}, \\ H, & \text{失败}. \end{cases}$$

为什么引入这个“失败惩罚长度”？如果只对成功任务求平均，失败率较高的算法可能仅凭少数快速成功样本获得看似漂亮的均值。这是典型的 **survivorship bias**。

另外，成功率与连续 $L$ 的统计形式不同：

- **成功比例**：报告 Wilson 95% 区间，避免接近 0% 或 100% 时普通正态近似不可靠。
- **均值 $L$**：用 Student-$t$ 区间，按执行种子成组汇总；由于 $L$ 可能呈双峰或高度偏态，报告承认区间是近似的。
- **主效应 / 消融效应**：在同一随机种子下计算差值，再求 seed block 的均值与区间，降低执行噪声。
- **跨实验组的旧/新 Guidance 比较**：使用 20,000 次 seed-cluster percentile bootstrap，整个 seed 对应的场景差值一起重采样。

### 为什么不能把 1,680 对当作 1,680 个独立样本?

同一个执行种子会在很多不同场景条件中被重复使用。把这些差值当作完全独立，会夸大有效样本量并使置信区间偏窄。

新版分析脚本先将样本按 seed 组织，然后重采样完整 seed cluster：

```
blocks[b['seed']].append(metric_of(b, metric) - metric_of(a, metric))
selected = rng.integers(len(values), size=(20000, len(values)))
draws = sums[selected].sum(axis=1) / counts[selected].sum(axis=1)
lower, upper = np.quantile(draws, [.025, .975])
```

它还从重复套件中删除 80 组条件，防止同一 Episode 在总体对比中被重复计权。

**这里是整份报告的统计亮点**。但区间仍然是针对固定实验布局和所设随机种子机制的推断，不是所有可能地图的保证。

## 八、两套规划预算——为什么不能混为一谈?

B3 正式 Demo：每步最大 10 秒，程序默认 planning_time=9.5，最多 2,000 Trial，哪个先到就停。

B4 主实验：每步允许很长的墙钟时间（仓库元数据记录 wall_clock_cap=3600），通常要求完成规定的 2,000 Trial，以便不同环境下控制**模拟工作量**。

这使 B4 更容易比较同样 Trial cap 的算法，却不能直接当作老师机器上的严格 10 秒表现。报告明确披露了这一区别。

你的 4 组发布 Demo 共做出 74 次真实决策，其中 73 次达到 2,000 Trial，上限受时间触发的只有 1 次（仍完成 1,815 Trial）。这说明在记录的机器与任务上，有限预算通常足以完成规定 Trial，但不保证硬件或未知场景变化后仍然如此。

## 九、如何把 B4 写成有说服力的研究故事?

用四层逻辑就够：

1. **问题**：联合动作空间指数增长，随机偏移与障碍使规划困难。
2. **方法**：单机 VI 作为强先验，联合 UCT/MCTS 负责真实联合随机耦合。
3. **验证**：S1–S5、消融、开发与评估分离、旧/新 Guidance 配对。
4. **边界**：VI 平均效率改善不等于所有成功率改善；在部分多机任务中，VI-Greedy 比有限预算 MCTS 更好。

**研究生课程真正奖励的是一个正确而有边界的结论，不是一个无论出现什么实验数据都宣称自己最优的故事**。

### 第五讲理解检验

- 为什么要在同一个种子下配对两种算法，而不只比较两个总体均值?
- 为什么 $|A|$ 从 36 到 676 时，不能立刻说“动作空间越大越好”?
- 为什么 2,000 Trial 不等于 2,000 次 simulate()?
- 为什么旧版 A 与新版 B 的对比需要删掉 80 组重复条件?

下一讲我们将把所有关键结果逐张图、逐组指标解释清楚。

# 第六讲：读懂新版 B4 的真实结果——VI 有效在哪里，MCTS 为什么未必更好?

本讲对照最新报告 B4、partB_components.tex、partB_details.tex、results/summary.json 以及各组 CSV。所有效果符号都遵守报告：若比较某种修改方案 B 对基线 A，则 $\Delta L = L_B - L_A$；$L$ 越小越好。

你在研究型课程里应该拥有一种基本能力：**看着一张含均值与置信区间的图，判断哪些话有证据，哪些话只是猜测**。

这次新版算法最有意义的收获，恰恰是一些「不符合直觉」的结果。我们分四层理解：复杂度、组件、VI/Dijkstra 对比、困难场景。

## 一、S1：三个复杂度因素，哪个最重要?

报告将 $L$ 定义为失败时按时域 $H$ 计入的惩罚完成步数，因此以下数值越小越好。

| 因素：高水平减低水平 | $\Delta L$ | 95% CI | 能下的结论 |
|---|---|---|---|
| 状态规模 $n_y$ : $5 \to 20$ | +0.52 | [−0.86, +1.90] | 当前设计中没有清楚的平均影响 |
| 动作 $\lvert A\rvert$ : $36 \to 676$ | **−3.83** | [−5.77, −1.88] | 更丰富动作在这些布局下缩短 $L$ |
| 障碍 0→5 | **+3.73** | [+2.55, +4.92] | 更多障碍明显恶化 $L$ |

S1 总共成功 **527/540 = 97.6%**；13 次失败全是碰撞（12 次障碍、1 次机间），没有因为用完 $H$ 而终止的失败。

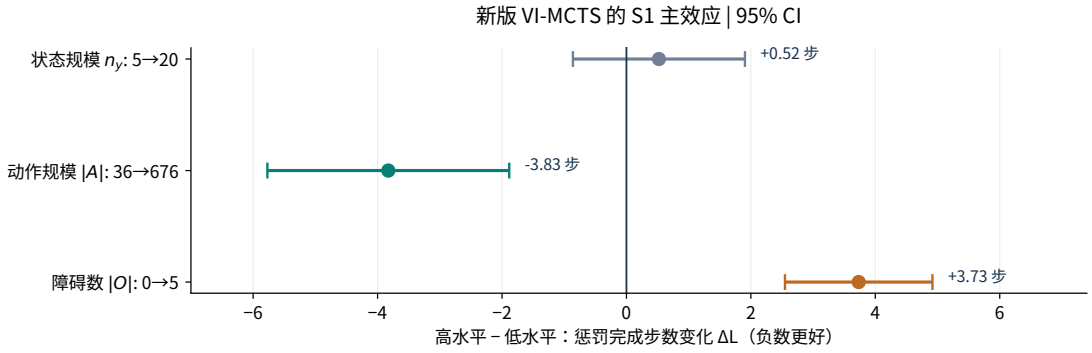



图 13: 新版 S1 主效应及区间

### 1. 为什么增大 State Space，没有明显退化?

你改变 $n_y$，增加了网格自由格子的数量，但仍然保持相近的有效路径构型。规划器在每个真实状态只搜索**当前可达附近的未来**，并不枚举整张网格的全部联合状态。

然而**不能**据此得出「MCTS 对状态空间增长不敏感」这样的广泛定理。实验主要扩大了与路线不直接相关的空间：两种大网格之间有大量样本轨迹完全一致。这反映了实验设计具体改变了什么，而不只是抽象的 $\lvert S\rvert$ 数字。

### 2. 为什么 Action Space 越大，反而可能更快?

26 个方向不仅使 $\lvert A\rvert$ 增大，还让无人机能使用更多对角或垂直位移。直觉上，在有障碍时，一步对角移动可能绕过一个必须分两步绕过的转角。

因此较大的动作集合代表更强**物理控制能力**，不是纯粹更多“搜索分支”。S2 用复制标签的实验控制了后者。

### 3. 为什么 Clutter 最明显?

障碍增加有双重影响：路径绕行（几何代价），以及偏移后撞障碍（随机代价）。对于 MDP，第二种代价还会改变未来期望价值，而不只是长度。

这里还有一个有意思的版本差异：上一版 Dijkstra 在相同 S1 设计下的障碍主效应约为 **+7.14**，最新版 VI 为 **+3.73**。这是本实验中的改善，不代表对所有复杂地图都如此。

### 4. 为什么没有一个因素的成功率 CI 排除零?

成功率本身接近天花板（97.6%），只有 13 次失败；不同因素产生的失败数差异较小，而成功二元指标方差较大。

报告给出：状态 −1.1 pp、动作 −1.1 pp、障碍 −2.2 pp，各自区间均包含 0。因此 $L$ **的效应比 Success Rate 更能区分这个具体 S1 实验中的细节**。

## 二、S2：控制实验告诉我们「为什么」，不只是「是什么」

### 1. 低噪声对照支持了障碍的双重机制

把 $\alpha$ 降到 0.01，障碍效应主要来自必须绕行；保持 $\alpha = 0.25$，还会叠加随机撞障碍的风险。

当前报告估计 Clutter 对 $L$ 的影响约为:

$$\Delta_C(\alpha = 0.01) \approx +1.0, \qquad \Delta_C(\alpha = 0.25) \approx +4.5.$$

这不能证明一个唯一的因果中介路径，但在模型机制和实验控制下，是有解释力的证据。

### 2. 动作标签复制验证了 Progressive Widening 的实际鲁棒性

$|A| = 676, 2704, 10816$ 时，任务成功率都是 19/20，平均 $L$ 大约 14.9、14.7、14.6；根节点实际纳入的动作数仍在 22–23 附近。

这符合渐进拓宽的意图：候选动作数量不是简单跟完整标签集合等比例增长。

但这些条件基本固定物理布局，**不能外推到更复杂的新物理运动方向**。

## 三、S3 消融：哪几个组件真正带来了收益?

下表都是「去掉该组件之后」减去完整版的效果；正 $\Delta L$ 意味着去掉后更差，所以完整版较好。

| 去掉的组件 | $\Delta L$ | 95% CI | 精确解读 |
|---|---|---|---|
| Uniform proposals/ rollouts（取消整套引导） | **+35.7** | [+33.6, +37.8] | 强引导不可或缺；同时移除了合法性过滤 |
| Zero leaf value | +0.5 | [−1.0, +2.0] | 未分辨出稳定的边际收益 |
| No progressive widening | +1.0 | [−0.5, +2.5] | 未分辨出稳定的边际收益 |
| No inter-drone risk term | **+1.8** | [+0.1, +3.5] | 有正向证据，主要集中在拥挤条件 |
| Most-visited final action | **+3.2** | [+2.1, +4.3] | Anchored Root 比最常访问动作更稳健 |
| Largest mean-$Q$ final action | **+3.7** | [+2.4, +5.1] | 直接选择样本均值最大动作也更差 |
| Guided policy, no search | +0.6 | [−1.3, +2.5] | 这组角点上搜索额外收益尚未确定 |

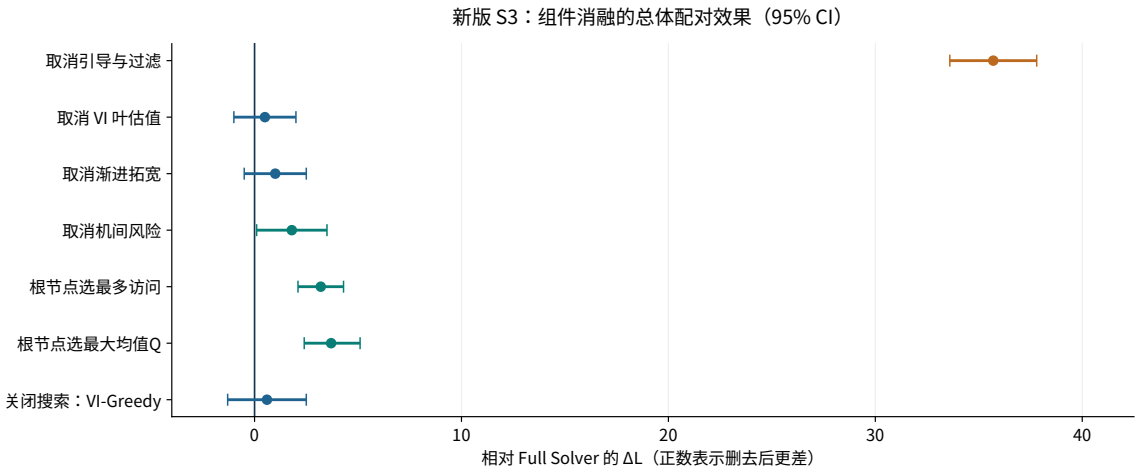



图 14: 最新版 S3 组件消融效果

### 1. 最大的增益来自 Guidance，而不是搜索树规模

no_guidance 一下让 $L$ 增加约 35.7 步，成功率降低约 50 个百分点。**但不能说「VI 单独贡献了 35.7 步」!** 这个消融同时关闭了目标导向的候选动作生成、Rollout 引导和意图落点合法性过滤。

要专门估计 VI 的额外贡献，我们必须看 **VI 与旧 Dijkstra 的成对对照**；下一节就是。

### 2. 为什么 Root Decision 有如此明显的影响?

UCT 选择规则存在探索奖励，根动作的模拟访问次数不一定代表最高的真实期望回报。若直接选访问最多，或者在有限采样下选最大样本均值，容易被偶然的随机回报误导。

Anchored Root 以强 VI-Greedy 为默认，并要求替代动作的回报估计超过一个基于方差的门槛，尽量避免轻率切换。

但门槛不是严格校准过的置信下界，因此不保证全局最优。

### 3. 不显著不等于完全没用

no_leaf 和 no_widening 的区间包含 0，只能说在这批场景、现有样本和预算下，**没有分辨出稳定边际差异**。不能推出它们在所有环境下都没用。

许多任务路线十来步就能完成，而搜索树和 Rollout 已经覆盖相近尺度；此时额外 Leaf Value 可能作用很小。对于更长路线、复杂协调场景，可能出现不同结果，但报告没有实验支持时不能当作事实。

## 四、最关键的正式比较：旧 Dijkstra A 对新版 VI B

这次算法优化要回答的真正问题，不是「VI 听上去更高级吗」，而是：**在相同场景、相同种子、相同搜索预算下，VI Guidance 是否有可测量的改善?**

报告 Appendix 的 Value-iteration versus Dijkstra guidance 表直接给出了答案。

| 实验组 | 配对数 | A 成功 | B 成功 | $\Delta L = B - A$（越负越好） |
|---|---|---|---|---|
| S1 factorial | 540 | 527 | 527 | −0.88 |
| S2 controls | 160 | 160 | 157 | +0.94 |
| S4 budget | 400 | 381 | 389 | −2.08 |
| S5 layouts | 160 | 148 | 144 | +1.12 |
| Regression | 360 | 275 | 273 | +0.19 |
| Supplementary（只计 Full） | 140 | 125 | 127 | −3.77 |

| | | | | |
|---|---|---|---|---|
| **合并（按条件去重，不含 baseline）** | **1,680** | **1,539** | **1,540** | **−0.82** |

**合并差值的统计区间：**

$$\boxed{\Delta L_{\text{VI}-\text{Dijkstra}} = -0.816, \qquad 95\%\,\text{CI} = [-1.380, -0.278]}$$

成功率差：

$$\boxed{\Delta P = +0.060\ \text{pp}, \quad 95\%\,\text{CI} = [-0.893, +1.061]\ \text{pp}.}$$

这里的 **pp（percentage points）**是「百分点」，不是百分比相对增长。

因此符合证据的表达是：**VI 在这些对照任务中改善了平均惩罚完成步数，但没有证据证明总体成功率也有显著提升**。

### 为什么行数加起来不是 1,680?

上表前六组总共 1,760 对，其中 80 对条件与其他套件重复。先删掉重复项，得到 1,680 组独立列出的条件对。

旧/新版比较表还另列 520 组「无搜索基础策略」的对比，但这些不加入 1,680 的总体估计，因为总体想比较的是**提交的完整求解器**而不是混入不同变体。

如果你在口试中能说清楚「重复条件」「配对实验」「分组 bootstrap」三个概念，就能证明你真的理解了报告，而不是背数字。

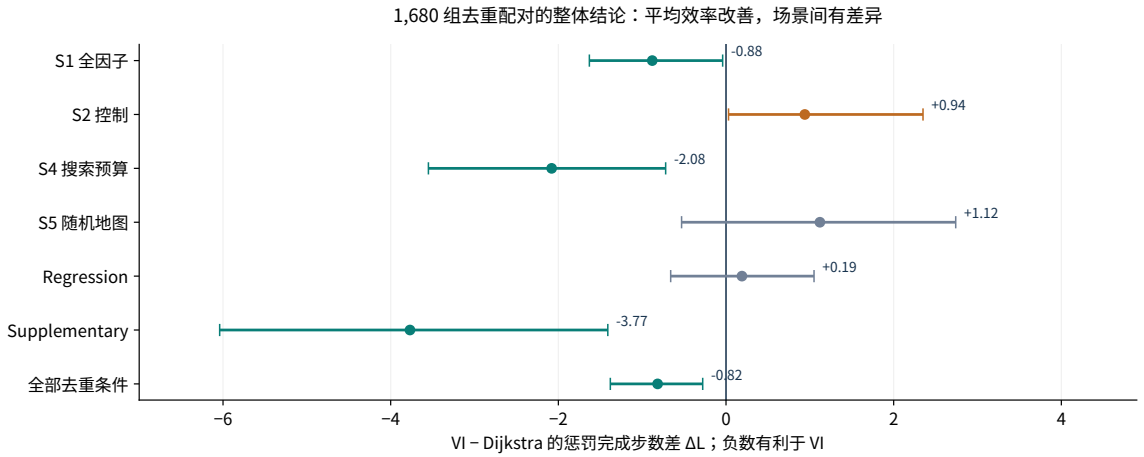



图 15: VI 和 Dijkstra 的分组效应图

### 这算“VI 优于 Dijkstra”吗?

在当前实验设计和指标 $L$ 上，可以说**平均结果支持 VI 的效率优势**；在成功率上不能做同样结论。

还有一个约束：开发集 Round 3 是根据成功数和平均 $L$ 选择了 VI，开发集提升存在选择偏差；正式评估与开发集分开，是为了减少这种偏差。报告对该限制已有披露。

## 五、为什么完整版 MCTS 有时反而不如 VI-Greedy?

这是整份 B4 最需要你诚实回答的**负面发现**。

从回归与生成场景抽出的 240 对**可行多无人机**运行：

| 指标 | VI-guided MCTS | 直接 VI-Greedy |
|---|---|---|
| 成功次数 | 221/240 | **229/240** |
| MCTS 独自挽救的 Episode | 5 | — |
| Greedy 独自挽救的 Episode | — | **13** |
| 移除 MCTS 后，惩罚完成步数变化 | — | **−2.36** 步，95% CI [−3.68, −0.94] |

这里的差值以「Greedy 减 Full」定义，**负数说明不运行搜索更好**。成功率差（Greedy − Full）约 +3.33 pp，95% CI [+0.44,+6.14] pp。

这说明在这些布局中，有限预算 MCTS 并没有自动优于它的强基线；甚至存在实证支持的基线优势。

为什么？至少有四种可能机制：

1. **强基线：**单机 VI 已经把很多短期随机决策计算得相当准确；
2. **稀疏联合后继：**多机随机结果组合太多，同一个子节点难以被反复访问，树浅；
3. **有限采样误差：**少量反常幸运/不幸的回报可能误导根动作判断；
4. **单机—联合近似冲突：**搜索不一定及时发现真正需要主动让行的协调策略。

但这是**解释假设**，不是 B4 已经逐项证明的因果机制。作为研究报告应该使用 may, consistent with, suggests，不要用 proves。

#### 一个“没有必要防御”的学术观点

**如果没有稳定的 MCTS 净收益，保留 MCTS 仍然符合 B2 的在线近似规划要求。** B2 让你设计可运行的具体求解器；B4 要求分析各组件效果，而不是预设每个组件必须提升成功率。

正确结论是：「当前 MCTS 框架可以采样联合碰撞与协调，但经验表明强单机 VI 基线解释了大量完成效果，联合搜索在这些预算和布局下增益不稳」。这比伪造一个“搜索永远更好”的故事强得多。

## 六、S4：为什么多试验 2,000 次也不总是更好?

| 场景 | 16 Trials 成功率 | 2,000 Trials 成功率 | 方向 |
|---|---|---|---|
| Open 2D（1 架） | 20/20 | 20/20 | 持平 |
| Strong wind（1 架） | 20/20 | 18/20 | **减少** |
| Four 3D（4 架） | 20/20 | 20/20 | 持平 |
| Factorial 两架复杂地图 | 18/20 | 19/20 | 小幅提升但存在噪声 |

不要得出“搜索越多越差”的结论：成功率的样本量只有每格 20，而且有些中间预算效果更好或更差。真正可说的是：**在这组 S4 场景中，没有建立搜索预算增加必然带来完成性能改善的单调关系。**

同时，四机搜索树的新节点平均深度随预算从约 2.13 增至 2.96，说明更多计算确实改变了树结构，但结构增长不等价于更好的最终动作。

## 七、S5 与压力场景：哪些数据发生了真实变化?

S5 合计 **144/160** 成功；四机 $\alpha = 2$ 的生成布局为 **6/10**，而不是上一版的 3/10。四机 $\alpha = 1$ 也是 6/10。

补充场景如下：

| 场景 | VI-MCTS 最新 | 旧 Dijkstra-MCTS | 要怎样理解 |
|---|---|---|---|
| 单机墙壁仅有一个缺口 | **14/20** | 13/20 | 最新成功者平均 19.6 步，旧版成功者约 38.6 步 |
| 单机曲折 Slalom | **20/20** | 19/20 | 当前布局有改善，但样本较少 |
| 双机共享走廊 | 13/20 | 13/20 | 成功率相同，仍有固有运动风险 |

| 场景 | VI-MCTS 最新 | 旧 Dijkstra-MCTS | 要怎样理解 |
|---|---|---|---|
| 四机三维标准场景 | 20/20 | 20/20 | 都很稳定，不能据此证明 VI 全面优越 |

特别需要提醒：上表中**成功轨迹的平均步数**是以成功为条件的统计，它与惩罚完成长度 $L$ 不是一个指标。某种方法成功率较低时，其成功者的平均步数可能反而偏小。

## 八、把本讲归结为三个有层次的结论

**结论 1（有正面证据）：** VI 比旧 Dijkstra 在综合配对条件上更能降低平均惩罚完成步数，尤其是部分预算与压力场景。

**结论 2（还不能证明）：** VI 提高总体成功率；MCTS 必定强于 VI-Greedy；更多搜索必然改善完成效率。

**结论 3（研究价值）：** MCTS 的扩展策略、最终根选择和强单机先验之间存在复杂关系；算法性能不仅由理论最优性决定，还由有限预算、采样方差与状态后继复用程度共同决定。

### 第六讲自测

1. 为什么「$\Delta L = -0.82$ 的 CI 不包含 0」不等于「成功率显著更高」?
2. 你会怎样解释新版 S1 障碍效应比旧版小，但某些单独回归场景却变差?
3. 为什么“VI-Greedy 在 240 对中领先”不是废除整个 MCTS 研究的理由?
4. 如果老师问 What explains the biggest gain?，你为什么不能只回答 MCTS?

下一讲从实验推进到数学边界：什么时候可以真正说“不可避免风险”，什么时候不能说“已经达到理论上限”。

# 第七讲：理论成功率上限、狭窄通道与“算法已经最优”的逻辑边界

本讲基于报告 Appendix「Supplementary end-to-end cases」、scripts/compute_ceilings.py、results/ceilings.json，以及老师提供的随机运动与终止规则。这里的「最优」特指**有限时域成功概率最优**，不是 B1 的期望折扣回报最优。

我们先回答一个经常被忽略的问题：**如果无人机必然存在随机偏移，那么一条通道里不管怎么规划都可能撞墙。是不是意味着失败已经达到了理论上限?**

回答是：第一句话可能正确，第二句话并不自动成立。你最新报告加入的 Exact Success Ceiling 诊断，就是为了在特定小场景中把两者严谨地区分开。

## 一、成功率最大化与折扣奖励最大化，不是同一目标

作业 B1 的目标:

$$\boxed{\max_{\pi} \mathbb{E}_{\pi}\Big[\sum_{t=0}^{\tau-1} \gamma^t r_t\Big]}$$

理论成功率分析的目标:

$$\boxed{\max_{\pi} \Pr_{\pi}(\text{所有无人机在} H \text{ 步内成功})}$$

如果奖励含时间惩罚、折扣和碰撞代价，一个完成速度更快但风险略高的策略，可能得到更高折扣奖励；而一个完成更慢但成功概率更高的策略可能最大化成功率。

因此一个策略即使在**成功概率指标**上接近最优，也不能据此证明它在**折扣奖励指标**上同样最优。

## 二、有限时域成功率 Bellman Backup

对于某个单机或小型联合状态 $s$，令

$$U_h(s) = \sup_{\pi} \Pr_{\pi}(\text{在剩余} h \text{ 步内成功} \mid s).$$

边界条件:

$$U_0(s) = \begin{cases} 1, & \text{已成功完成,} \\ 0, & \text{否则.} \end{cases}$$

对于未终止状态且 $h \geq 1$:

$$\boxed{U_h(s) = \max_{a} \sum_{s'} P(s' \mid s, a) U_{h-1}(s')}$$

其中碰撞终止的后继状态成功概率设为 0，成功终止状态设为 1。

与奖励 VI 相比，**方程形状相同，但边界与优化目标不同**。这非常重要，考官经常利用这种相似性考你是否真正理解 Bellman 动态规划。

当状态空间很小、时间有限时，可以枚举所有相关状态和动作，算出理论上任何策略可达到的最大成功概率。

## 三、墙壁缺口：理论最优约 70.5%，当前算法 14/20

项目 ceilings.json 对单机墙壁缺口给出：

$$\boxed{U_H(s_0) \approx 0.70519025.}$$

实际提交 VI-MCTS 在 20 种子下成功：

$$14/20 = 70\%.$$

两者数值非常接近，但请谨慎理解：**20 次随机执行的样本比例非常不稳定**。其 Wilson 95% 区间约为 [48%,85%]，包含理论上限，因此数据与最优成功率相容，但不能据此证明策略已经达到理论最优。

为什么必须保留这个区别？因为你很容易构造一个真实成功率只有 60% 的策略，在 20 次随机样本里碰巧成功 14 次。样本点接近上限，不代表策略本身等于上限。

在这条墙壁缺口任务中，静态最短路线需要 11 步，而 VI-MCTS **成功的轨迹**平均约 19.6 步。上一个 Dijkstra 版本成功轨迹平均约 38.6 步。这是新版减少犹豫、缩短危险区暴露时间的有价值迹象，但它仍然没有证明具体哪个修改导致改善。

## 四、共享走廊：理论最优约 63.1%，当前算法 13/20

项目 ceilings.json 对两架无人机、目标位于共享通道中的场景给出：

$$\boxed{U_H(s_0) \approx 0.63085334.}$$

提交 MCTS 在 20 种子下成功 13/20=65%。注意，**样本成功率可以高于理论最优值**，因为一个有限样本的频率允许随机波动。这不是理论上限有错。

报告同时通过老师的仿真器执行了理论成功率最优策略的大量独立采样：原始 2,000 次加上额外 8,000 次，共 10,000 次，合计成功率约 63.0%，与计算结果相符。

**如果把 13/20 误当成算法实际成功概率并宣称“超过理论上限”，就犯了把样本估计当作真实参数的错误。**

共享走廊还有一个单机无法表达的结构：先到目标的无人机会冻结且继续占格，可能堵住另一架的必经路径。单机 VI 在这个问题上只是一个放松；联合成功率的 DP 才能精确处理两架无人机的相互限制。

## 五、四机高噪声：为什么不能计算完整联合理论上限?

当四架无人机处于 $8 \times 6 \times 3$ 的空间时，约 140 个有效位置组成的联合位置空间就已经是：

$$140^4 \approx 3.84 \times 10^8.$$

这还没有乘上 reached flags、时间步以及四机联合动作集合。完整枚举联合 Bellman Backups 的成本非常高。

因此你的报告**没有宣称已经计算四机联合精确最优成功率**。

报告的另一种诊断是：先让每一架无人机单独最优规划，再同时执行这些相互独立的单机策略。

对于 random_15_N4，每架无人机独自完成的理论成功率均至少约 94%，然而把这些不协调的单机最优策略一起执行，在 1,000 次诊断试验里只有约 **16%** 联合成功；提交 VI-MCTS 在该具体布局的 10 次测试中达到 **6/10**。

这强烈提示**协调与随机碰撞耦合很重要**，但后两组样本量、实验方法均不同，不能把它当成严格的同种子因果对比，更不能从中推出 MCTS 已达到最优。

## 六、一个容易考错的概率推论：单机成功率高，联合成功率不一定高

即便四架无人机的单机成功概率都是 $p = 0.94$，在完全独立、彼此不影响的理想条件下，四架都成功的概率也只是：

$$0.94^4 \approx 78.1\%.$$

而真实系统里，四架无人机的成功事件并不独立：它们可能随机进入同一格，或提前冻结在他人的路径上。因此不能直接把单机成功率相乘当作真实系统的联合成功率。

**更重要：上面 78.1% 是理想化教学计算，不是 random_15_N4 的模型上限。**

## 七、为什么风险公式会造成“方向不敏感”的现象?

从某个内部自由格子 $p$ 出发，假设所选意图方向合法、且不是障碍，随机落入障碍的质量只来自其他方向的偏移。

$$\boxed{\rho(p) = |\mathcal{N}(p) \cap O|\, \sigma(p).}$$

对于所有**合法的自由意图目标**，在固定当前位置 $p$ 时，这项**当步障碍撞击概率**不随意图方向改变：一旦目标不是障碍，障碍方向都落在“其他方向”集合里。

但选择不同方向会落到不同的后继状态，造成**未来**风险差异，所以「当步风险与方向无关」绝不等于「所有动作价值相同」。

这也解释了墙壁缺口场景为什么可能出现犹豫：即使向安全方向后退，也不一定降低此时立刻遭遇随机撞墙的概率；反复靠近墙壁则会增加在危险邻域停留的时间。

## 八、三类失败机制必须分开讨论

| 类型 | 主要障碍 | 计算上限情况 | 可以得出的结论 |
|---|---|---|---|
| 单机墙壁缺口 | 仅有一条跨墙通道，随机偏移不可避免 | 单机精确成功率 DP 已完成 | 策略不可能 100% 成功；当前 14/20 与理论值相容 |
| 双机共享走廊 | 运动随机性 + 目标冻结后的占路协作 | 特定双机模型的联合成功率 DP 已完成 | 13/20 与最优值相容，但仍不是策略最优性证明 |
| 四机强噪声 | 联合后继规模、机间冲突与高噪声 | 未计算完整四机联合上限 | 6/10 只是有限样本表现，不能声称达到上限 |

注意报告还明确指出：墙壁缺口与共享走廊分别使用 7/8 个障碍物，**超出 B3 描述的最多 5 个障碍物测试范围**。它们是诊断性的压力测试，不应拿来夸大或贬低正式 Demo 的通过概率。

## 九、什么叫“证据有力，但还不构成证明”?

假如你的上限 DP 计算结果是 $U_H = 0.705$，某个策略执行了 20 次，成功 14 次。

合理说法:

> The observed success rate is compatible with the finite-horizon success optimum of the supplied model under the reported uncertainty.

不合理说法:

> Our MCTS is provably optimal because its sample success rate matches the theoretical bound.

这是科学推断的严肃性：**理论值、有限样本观察值、优化目标与因果解释，必须分开**。

## 十、局限性与后续研究方向：可以提，不要假装已经做过

你报告列出的实际限制包括：

1. S1 的状态变化主要是 off-route padding，并不等于长路径规模实验；
2. S5 每个 $(N, \alpha)$ 只有一个布局，随机性与特定布局高度耦合；
3. 主实验规定 Trial 数量，但没有用 B3 的严格墙钟时间截止，因此不能用 B4 耗时证明任何硬件都满足 10 秒；
4. 单机 VI 忽略他机未来动作，机间风险只通过提案近似；
5. MCTS 随机后继分散，四机状态可能难以复用，树仍浅；
6. CI 是逐点区间，没有做多重比较校正；部分双峰数据的 $t$ 近似需要谨慎。

报告还提到了 Outcome Progressive Widening 或 Open-loop Tree 可能提高随机后继复用。**这属于未来工作建议，没有实现，也没有评价实验**。

### 第七讲总结

对困难机器人规划任务，一个成熟的回答应该是：

**我能够指出哪些失败源自模型内在的不确定性，哪些可能源自有限预算与弱协调；对小任务可以精确计算成功率上限，对四机大任务则诚实承认计算与证据的边界**。

如果你能把「成功概率最优」和「折扣奖励最优」严格区分，你对这份作业的理论理解就已经超过仅会调用 MCTS 的程度。

# 第八讲：像机器人规划研究者一样解释项目——代码地图、口试问答与证明边界

本讲把前七讲收束为一套可以在 COMP8620 B3 Demo 与 oral Q&A 中完整表达的知识框架。所有具体文件名、参数和结果与 commit 5c7e9c20 对应。

到这里，我们已经从模型推导到单机 VI，再到联合 MCTS、代码执行与研究实验。现在需要练习另一种能力：**不依赖讲义，你能否独立回答为什么如此建模、为什么如此实现、数据到底支持哪些结论?**

## 一、把整个系统浓缩成一张因果结构图

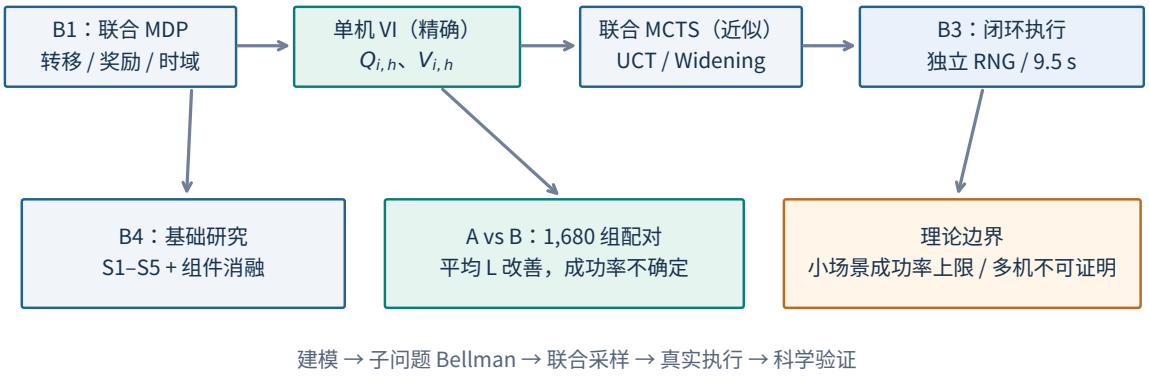



图 16: 新版 Part B 系统总图

主线其实只有一句话:

> 已知多无人机的随机运动 MDP，但联合最优规划不可承受；先对各个单机放松模型做有限时域 VI，得到可靠的 $Q_{i,h}$ 与 $V_{i,h}$，再用它引导联合 UCT 搜索；在真实执行的每一步重新规划，最后用控制变量实验评价其效果与局限。

## 二、代码阅读地图：从上到下应该怎么看?

建议先按**依赖关系**而不是文件长度读代码。

| 文件/函数 | 关键职责 | 你应该记住的代码特征 |
|---|---|---|
| multi_drone.py | 老师随机转移与奖励 | step / simulate、越界归一化、冻结目标、同格碰撞 |
| run_u8076655.py | 创建执行环境、模拟环境、规划器 | 三套 RNG、run_episode、统计日志、单步墙钟预算 |
| planner.py:__init__ | 地图预计算 | BFS、Dijkstra 回退、_value_iteration() |
| planner.py:_slip_model | 单步随机运动概率 | intended mass、slip mass、局部障碍风险 |
| planner.py:_value_iteration | 单机 Bellman Backup | csr_matrix、Q = reward + gamma*(P @ V)、逐时域历史 |
| planner.py:_value_options | 由 $Q_{i,h}$ 导出动作分数 | −q / risk_unit + tie，过滤意图目标 |
| planner.py:_proposal | 顺序构造联合动作 | 预约目标格、softmax Sample、随机 Explore、机间风险 |
| planner.py:_search | UCT / progressive widening | 动作选择、随机后继、递归、回报回传 |

| 文件/函数 | 关键职责 | 你应该记住的代码特征 |
|---|---|---|
| planner.py:_rollout | 未展开未来的模拟策略 | Guided Proposal、探索比例 0.1、深度 8 |
| planner.py:_leaf | 截断时的联合价值代理 | 单机 VI 价值加近似的 reached 成本 |
| planner.py:_anchored | 最终动作决策 | VI-Greedy 为锚，估计值差足够大才切换 |
| scripts/analyse_results.py | B4 统计与图表 | 配对、种子聚类 bootstrap、重复条件去重 |
| scripts/compute_ceilings.py | 特定小场景成功概率上限 | 不是完整四机联合最优策略求解 |

不要把 _value_iteration() 和 _search() 混为一谈。**前者对单机有限状态做完整 Bellman 递推，后者在当前联合状态上随机采样未来**。

## 三、最重要的 20 个口试问题（附参考回答）

### Q1. 为什么这里是 MDP，而不是普通最短路?

因为命令动作的实际后继是随机的，且碰撞与目标奖励取决于后继联合状态；最优决策应比较期望累计奖励而非仅仅几何步数。状态包括位置、到达标志和剩余时域，保证 Markov 性。

### Q2. 已经知道随机转移模型，为何不直接对整个联合系统做 Value Iteration?

联合位置数量大致随 $F^N$ 增长，动作数量为 $B^N$，还要考虑 reached flags、时域和随机后继。对四机计算完整转移矩阵和 Bellman Backups 成本过高；但单机子问题相对可解，因此单机 VI 与联合在线搜索具有合理的计算分工。

### Q3. Value Iteration 不是一个完整算法吗? 为什么还需要 MCTS?

单机 VI 计算 $V_{i,h}(p)$ 时忽略了其他无人机的位置、未来动作、碰撞和目标占用；完整联合 MDP 的价值不是单机价值直接求和。联合 MCTS 使用原始生成模型采样这些耦合，但在有限预算下只能得到近似估计。

### Q4. 为什么有两个环境实例?

execution.step() 推进真实任务；model.simulate() 在规划器内部采样假想未来。二者使用相同的转移规律，但独立 RNG，保证规划模拟不会改变真实执行的随机数序列。

### Q5. 为什么还要第三个规划器 RNG?

它单独控制动作提案的随机性。这样执行扰动、生成模型采样与搜索的探索采样不会互相污染随机数序列，提高重复实验与配对比较的可解释性。

### Q6. 单机 VI 的 Bellman 方程是什么?

$$Q_{i,h}(p,a) = r_i(p,a) + \gamma \sum_{p'} P_i(p' \mid p,a) V_{i,h-1}(p'), \quad V_{i,h}(p) = \max_a Q_{i,h}(p,a).$$

碰撞分支的未来价值为零，到目标后也令未来价值零。真实动作成功概率按边界归一化。

### Q7. VI 与旧 Dijkstra 关键差别是什么?

Dijkstra 在近似确定性图上优化固定的距离与局部风险成本；VI 直接根据老师的随机运动分布、碰撞惩罚、目标奖励和剩余时域做 Bellman 更新，因此获得有时间索引的 $Q_{i,h}$ 和 $V_{i,h}$。但是 VI 仍是单机放松，不是联合最优。

### Q8. 为什么还需要 Dijkstra?

它是历史基线，且在 VI 状态/动作/时域存储规模超过阈值时作为回退。并非最新版的默认算法。

### Q9. 为什么单机 VI 的 $Q$ 值要取负号?

Bellman $Q$ 越大越好，_proposal() 用越小越好的候选成本排序；$-Q/\eta$ 将价值转换成同尺度的代价，距离同分项和机间风险再叠加。

### Q10. 为什么不能完全依赖 VI-Greedy?

在一些联合任务中，主动让路或避开同伴的目标格可能要求单机价值短期降低，以换取联合成功。VI-Greedy 不能表示所有联合耦合；MCTS 有可能通过采样评估这样的协调动作。但有限预算下它不保证更好。

### Q11. UCT 的 Exploration 与 Exploitation 怎么平衡?

$$a^* = \arg\max_a \left[ Q(s,a) + c\sqrt{\log(N(s)+1)/N(s,a)} \right].$$

已访问很多的高均值动作有利用价值，访问很少的动作有探索奖励。这里 c=0.35，而不是根节点最终执行规则的 switch_z。

### Q12. Progressive Widening 的意义?

四机三维动作共有 $26^4 = 456{,}976$ 种。根节点最多约 $\lceil 0.5\sqrt{N+1} \rceil$ 个动作，在 2,000 Trial 时约 23 个，可以把有限采样集中在有指导意义的候选上。它不提供全空间完整性保证。

### Q13. MCTS 的四阶段如何与代码对应?

Selection：优先未访问动作，再用 UCT；Expansion：渐进纳入 _proposal() 生成的联合动作；Simulation：新随机后继用 Guided Rollout 与 VI Leaf 估值；Backpropagation：折扣回报样本更新 Edge 的均值、方差及访问数。

### Q14. 为什么 Rollout 和 Leaf 都存在?

树内搜索深度只有 8，Rollout 额外最多 8 步；如果仍未终止，必须估计剩余价值。新版 Leaf 利用单机有限时域 VI，不再主要依赖旧版 BFS 乐观估计。

### Q15. 为什么不用 Q-learning 来更新树边?

你已有准确的生成模型，当前做的是在线规划的 Monte Carlo Return Backup。边均值直接来自模拟轨迹回报。Q-learning 是另一种 TD 学习机制，其目标和训练过程不同；题目没有要求学习一个离线 Q 网络。

### Q16. 为什么根节点不直接选最大 $Q$?

因为有限采样会产生估计噪声。_anchored() 以 VI-Greedy 为默认，要求替代动作在估计均值上超过一个方差相关门槛，才考虑切换。它是稳健选择的启发式，不是严格统计证明。

### Q17. 旧 A 和新 B 的对比支持什么?

1,680 组去重配对中，VI 比 Dijkstra 平均降低惩罚完成步数约 0.82 步，95% CI [−1.38,−0.28]。成功率只增加约 0.06 个百分点，95% CI 包含 0，所以**不能宣称成功率显著提升**。

### Q18. 为什么 MCTS 有时候还不如 VI-Greedy?

强基础策略已经足够好，而联合随机后继非常分散，有限 Trial 难以充分复用树节点；采样方差可能导致根动作切换错误。240 对可行多机任务里 VI-Greedy 成功 229 次，Full 只有 221 次。报告如实承认，这不否定在线 MCTS 框架的合法性。

### Q19. 为什么 B4 允许固定 2,000 Trial，而 Demo 必须 10 秒?

两者回答不同的问题：B4 用固定 Trial 控制计算工作量；B3 检验严格墙钟限制下的运行能力。报告已经区分两种预算，不能用放宽墙钟限制的 B4 数据直接证明任意机器上的 10 秒 Demo。

### Q20. 70.5% / 63.1% 理论上限说明什么?

它们是指定小场景按**最大成功概率**计算的有限时域动态规划结果；说明一定存在不可避免的 slip 风险。MCTS 的 14/20 和 13/20 观察与这些上限相容，但并没有证明奖励最优，也没有证明 MCTS 必然达到上限。四机高噪声未计算联合精确上限。

## 四、给考官的 90 秒英文技术介绍（示范文本）

> I formulate the multi-drone task as a finite-horizon MDP with stochastic motion, collision penalties, and goal rewards. Full joint dynamic programming is computationally expensive because the action space grows as $B^N$ and stochastic joint outcomes spread across many states. My implementation first solves each drone's relaxed MDP by finite-horizon value iteration, using the supplied transition law and reward model. The resulting state-action values guide joint action proposals and Monte Carlo rollouts, while a centralised UCT tree samples the original joint dynamics. Progressive widening limits the number of admitted actions, and an anchored root rule retains the strong greedy base action unless search estimates favour switching. I then compare the final solver against the earlier Dijkstra guidance using matched seeds, factorial complexity experiments, component ablations and confidence intervals. Value guidance improves penalised completion length on average, but I do not claim a statistically established success-rate gain or a universal advantage over the guided base policy.

这段话的重点不是炫耀技术名称，而是用「问题—可计算的放松—联合近似—实验结论—边界」来组织表达。

## 五、必须能背后理解而非机械背诵的公式索引

| 概念 | 核心公式 | 你应该怎样说 |
|---|---|---|
| 状态 | $s = ((p_i, r_i)_{i=1}^N, t)$ | 位置、是否抵达、当前时域 |
| 联合动作 | $\lvert A\rvert = B^N$ | 集中式动作空间指数增长 |
| 意图概率 | $\pi(p) = d(p)/(d(p)+\alpha)$ | 运动模型的原始成功权重 |
| 越界归一化 | $\ell(p) = \pi(p)/M(p)$ | 过滤越界后再归一化 |
| 单机 VI | $Q_{i,h} = r_i + \gamma P_i V_{i,h-1}$ | 较小随机子问题做精确 Bellman |
| 单机状态价值 | $V_{i,h} = \max_a Q_{i,h}$ | 保留剩余时域信息 |
| Guidance score | $-Q_{i,h}/\eta + \text{tie} + \text{risk}$ | 用单机价值排序联合动作候选 |
| UCT | $Q(s,a) + c\sqrt{\ln(N(s)+1)/N(s,a)}$ | 搜索内部平衡探索与利用 |
| Progressive Widening | $K \approx \lceil 0.5\sqrt{N+1}\rceil$ | 控制可纳入搜索的动作数量 |
| Rollout | $G = r_0 + \gamma r_1 + \cdots + \gamma^L V_{\text{leaf}}$ | 采样未展开的未来 |
| 经验均值 | $Q \leftarrow Q + (G-Q)/n$ | Monte Carlo Backup |
| 根切换 | $Q(a) - Q(a_g) > z\hat{\sigma}\sqrt{1/n_a + 1/n_g}$ | 不轻易放弃强 VI-Greedy |
| 惩罚完成长度 | $L = \text{steps if success else } H$ | 失败也必须计入指标 |
| 成功率上限 | $U_h(s) = \max_a \sum P(s'\lvert s,a)U_{h-1}(s')$ | 与折扣奖励最优目标不同 |

## 六、十道真正考察理解的练习题（附要点）

**练习 1：** 既然 $\alpha = 0.01$ 时几乎按意图方向运动，为什么两个方向依然可能有不同 VI 价值?

要点：不同位置对应不同未来路径、折扣奖励、边界归一化和到目标时域。

**练习 2：** 为什么 P 稀疏矩阵的每行不一定和为 1?

要点：只存非终止自由后继，撞障碍的概率被单独作为终止质量。

**练习 3：**如果 V[g]=0，是否意味着到达目标没有奖励?

要点：奖励在进入目标的那次动作中支付，终止后未来价值为零，避免重复奖励。

**练习 4：**为什么四机用逐机排序可能错过最佳联合动作?

要点：局部最佳动作组合不一定全局最佳，目标占用与让路需要联合协调。

**练习 5：**为什么 no_risk 不代表 VI 丢失了所有障碍风险意识?

要点：取消的是额外机间风险代理项，单机 VI 转移中的障碍碰撞概率仍然存在。

**练习 6：**如果 MCTS 看到一个平均 $Q$ 更高的动作，为什么仍可能返回 VI-Greedy?

要点：Anchored Root 要求越过方差相关阈值；样本均值噪声大。

**练习 7：**为什么 B4 固定试验次数的数据可以帮助比较算法，但不能代表严格的每步 10 秒性能?

要点：同样 Trial cap 仍可能花费不同墙钟时间；10 秒真实截止可能截断搜索。

**练习 8：**为什么 S1 动作效果为负不是性能恶化?

要点：定义的是高水平减低水平的 $L$，越低越好。

**练习 9：**如果 VI 与 Dijkstra 的成功率差 CI 包含 0，但 $L$ 差 CI 不包含 0，你应该如何写结论?

要点：平均效率改善有证据，成功率提升未分辨。

**练习 10：**已知墙壁缺口理论成功率上限为 0.705，而某策略 20 次成功了 15 次，这是否推翻 DP?

要点：不推翻。有限样本频率允许超过真实成功概率和理论策略上限。

## 七、三层理解标准

**能复述：**你可以说出 MDP、Bellman、UCT 的定义与四阶段。

**能解释：**你可以把公式逐项对应到真实代码，知道 VI 为什么比 Dijkstra 多了随机转移和时域信息。

**能批判：**你可以指出采样的有效预算、配对的统计假设、强 VI-Greedy 与 MCTS 的竞争关系，以及理论上限证明范围。

真正的研究生水平在第三层。你不必保证你的每次规划都是最优，但应当能证明你知道**什么东西已经被验证，什么东西只是合理设计，什么东西还没有证据**。

### 第八讲总结

**这份作业的完整知识路线是：精确定义随机模型 → 精确求解较小的单机放松 → 用它指导有限预算联合 MCTS → 在真实环境闭环执行 → 用可复现统计实验发现收益与局限。**

能把这五个箭头说清楚，比背诵一百行伪代码更有用。

# 附录 A：原题要求、课程出处与项目版本

本讲义以 **COMP8620 Assignment 2 Part B（35 分）**、课程 Week 5–6 的 MDP 在线求解材料，以及项目代码与报告为主。理论解释尽量保留课程的术语；新增的数学算例明确作为教学演算，不应误当作实验记录。

| 子题 | 分值 | 主要要求 | 本讲义 |
|---|---|---|---|
| B1 | 3 | 给出状态、动作、转移、奖励和目标的 MDP 模型 | 第一讲 |
| B2 | 4 | 具体的在线近似求解器及伪代码；MCTS 四阶段 | 第二、三讲 |
| B3 | 8 | 真实 Python 实现、现场演示；每步不超过 10 秒、无并行搜索 | 第四讲 |
| B4 | 20 | State/Action/Clutter 系统变化；多随机种子、均值、95% CI；组件影响 | 第五、六、七讲 |

**原题与课程文件：** /mnt/data/a2.pdf；week5–MDPSolving.pdf；week6–MDPSolvingCont.pdf；MCTS.pdf；OnlineMDPSolving–RTDP–LRTDP.pdf。其中 Bellman Backup、UCT、MCTS Selection/Expansion/Simulation/Backpropagation 直接属于课程规划内容；Progressive Widening、VI 作为动作提案及根节点锚定规则是针对作业的具体实现设计。

**核对的仓库版本：** lrlorry/8620, main, commit 5c7e9c20c82e9ce2a65e5b90e9d871160ee7a915（2026-10-25）。

- 正式报告 Part B
- Part B 附录
- 默认 VI-MCTS 规划器
- 可执行入口
- 老师随机运动模型
- 实验原始汇总
- 分析脚本
- 最新发布验证
- 成功率上限

**版本说明：** 本讲义与此前 60 页旧版的主要区别在第三讲（VI 的完整推导和源码）、第四讲（最新版 Demo 与 45 项测试）、第五六讲（3,460 次新实验与 VI/Dijkstra 配对）、第七讲（精确成功率上限），以及所有相关公式与结果表。旧版默认 Risk-aware Dijkstra 只作为历史对照和大型地图的回退方案。

**证据边界：** 讲义中的训练/评估数据、具体参数和代码行为来自上述提交。教学算例（例如计算一个假设动作的 $Q$）仅用于概念教学。对原因的解释若报告没有独立隔离实验，会明确采用「可能」「与……一致」等审慎措辞。

# 附录 B：逐项学习检查清单

- ☐ 能写出 $s = ((p_i, r_i), t)$，并解释终止吸收状态。
- ☐ 能说明 $B^N$ 联合动作复杂度，而不是把四架机器人的最短路分别当作最终答案。
- ☐ 能解释越界方向被删除、障碍落点参与碰撞判定的区别。
- ☐ 能根据 $d/(d+\alpha)$ 推导单步运动概率并做边界归一化。
- ☐ 能解释单机 VI 的目标奖励只在首次到达时支付。
- ☐ 能写出 $Q_{i,h} = r + \gamma PV_{i,h-1}$ 并解释 $h$ 的意义。
- ☐ 能说明 csr_matrix 存的是什么、碰撞概率为什么单独处理。
- ☐ 能分辨默认 VI Guidance 与旧 Dijkstra 回退。

- □ 能解释 −q/risk_unit、预约目标格、机间风险附加项。
- □ 能区分 UCT Selection 和最终 Anchored Root Choice。
- □ 能解释 Progressive Widening 为什么重要，但不能保证找到全局最优。
- □ 能解释 Guided Rollout 与 VI Leaf 的区别。
- □ 能区分真实执行 RNG 与模型模拟 RNG。
- □ 能区分 9.5 秒规划预算、2,000 Trial 和 generative calls。
- □ 能复述 S1–S5 的实验设计与不同的因果限制。
- □ 能理解 $L$ 对失败 Episode 按 $H$ 惩罚的统计理由。
- □ 能解释 95% CI、成对 seed、20,000 次 seed-cluster bootstrap。
- □ 能重述 VI 与 Dijkstra 的 1,680 组总体结果，正确区分 $\Delta L$ 和成功率差。
- □ 能解释 VI-Greedy 在一些多机任务优于完整 MCTS 的事实。
- □ 能说明 70.5% / 63.1% 是小场景成功率 DP 上限，不是 MCTS 奖励最优证明。
- □ 能指出报告的 45 个单元测试、3,460 次正式评估和 2,880 次开发测试用途不同。

---

课程讲义终。主要用于理解 COMP8620 A2 Part B 的模型、算法、代码与实验逻辑，不替代原始报告、老师的课件或最终提交包。